

AI-Powered Business Development Radar: Automated Signal Detection and Classification from Heterogeneous Sources Using Large Language Models

Project study at the Chair for Business Analytics and Intelligent Systems in cooperation with Plan.Net TechNest (Serviceplan Group) at the Technical University of Munich

Supervisor: Paul Bischoff

Chair: Business Analytics and Intelligent Systems

Submitted by:

Name	Matriculation Number	Email Address
Valeria Vaquero Sáez	03802574	valeria.vaquero-saez@tum.de
Franziska Stannek	03787587	franziskasophia.stannek@tum.de
Olaf Tackowski	03793588	olaf.tackowski@tum.de
Azra Özçelik	03786156	azra.oezcelik@tum.de
Can Öcek	03757074	ge47tev@mytum.de

Date of Submission: 31.05.2026

Table of Contents

1. Introduction	4
1.1 Background & Motivation	4
1.2 Problem Statement.....	5
1.3 Objectives of the Project	5
1.4 Scope & Constraints.....	6
1.5 Structure of the Paper	7
2. Literature Review.....	7
2.1 Nature of Web Data and Its Challenges	7
2.2 Approaches to Extracting Value from Web Data	8
2.3 Web Scraping as a Data Collection Method.....	8
2.4 Types of Web Scraping Techniques.....	8
2.5 Legal and Ethical Constraints of Web Data Collection	9
2.6 Processing and Structuring Extracted Web Data	9
2.7 Large Language Models for Business Insight Generation	9
2.8 Prompt Engineering for Consistent Output.....	10
3. Legal Assessment Process	10
3.1. Purpose and Assessment Criteria	10
3.2. Source Assessment Procedure.....	11
Table 3.1. Six-Step source assessment procedure	12
3.3. Risk Classification.....	12
3.4. Platform Restricted Sources based on the Example LinkedIn.....	12
3.5 Alternative Access Approaches.....	13
3.6 Technically Restricted Sources and Responsible Access.....	13
3.7 Final Source Decision	14
4. High-Level System Overview & Architecture	14
5.0 Data Sources and Data Preparation	16
5.1 Data Sources	16
5.2 Scraping Strategy & Methods.....	18
5.2.1 Structured Source Discovery: RSS Feeds and Sitemaps	18
5.2.2 Unstructured Source Discovery: Recursion	19
5.2.3 PDF-Oriented Crawling	19
5.2.4 Dynamic Source Access with Playwright	19

5.2.5 Dynamic Source-Specific Page Interaction	20
5.2.6 Filtering and Data Quality	20
5.2.7 Responsible Access and Identification	20
5.2.8 Summary of Configurations per Source.....	21
5.3 HTML Extraction.....	21
5.3.1 Trafilatura	21
5.3.2 CSS selectors.....	22
5.4 Database Persistence and Deduplication of Scraped Sources	22
5.5 API calls to North Data	23
5.6 PDF Processing.....	23
6. LLM-Based Signal Classification	25
6.1 Rationale for Using Large Language Models.....	25
6.2 Signal Taxonomy and Classification Dimensions	26
6.3 Prompt Design and Output Structure.....	27
6.4 Ambiguity, Edge Cases and Consistency Measures	29
6.5 Cost Awareness and Scalability Considerations	30
7. Vector Storage and Retrieval.....	31
7.1 Structured Metadata Retrieval	31
7.2 Vector Search.....	31
7.3 Vector Embedding Generation and Storage	32
8. Pattern Analysis & Company-Level Reasoning	32
8.1 Why Isolated Chunk-Level Signals Are Not Sufficient.....	32
8.2 Company-Level Aggregation of Retrieved Signals	33
8.3 LLM2-Based Synthesis and Final Output Generation	34
8.4 Remaining Challenges of Company-Level Reasoning	35
9.0 User Interface and Interaction Concept	35
9.1 Core User Flows	36
9.2 Prototype Structure and Screens	37
10. Evaluation & Results	40
10.1 Quantitative Evaluation	40
10.1.1 Data Collection Results	40
10.1.2 LLM1 Classification & Categorization Results	41
10.1.3 Storage Results	43
10.2 Qualitative Evaluation	44
10.2.1 ECE Group: Confirming a Market-Level Pattern from Hiring Signals	44

10.2.2 Olympus: Surfacing a Recently Recognized Opportunity	45
10.2.3 General Feedback	45
10.3 Error Analysis	46
10.4 Limitations	47
10.4.1 Prompt Sensitivity	47
10.4.2 Cost Constraints	47
10.4.3 Legal Constraints	47
10.4.4 Evaluation Scope.....	48
10.5 Conclusion	48
10.6 Future Work	48
11. Summary of Contributions.....	50
12. References	54
13. Appendix.....	59
Appendix A: Recursive Web Crawling Flowchart.....	59
Appendix B: Playwright Item Interaction Logic.....	59
Appendix B.1: Paginated Link Discovery Flow.....	59
Appendix B.2: Load More and Accordion Expansion Flow	60
Appendix B.3: Investor PDF Collection Flow	60
Appendix C: Deduplication Logic.....	60
Appendix D: Evaluation Details.....	61

1. Introduction

This chapter introduces the motivation, problem setting, objectives, scope, and structure of the project. It begins by describing the practical challenges of business development in information-rich but unstructured environments and the opportunity that AI-based approaches present in addressing these challenges. We then define the problem this project addresses precisely, followed by a statement of the project's objectives and an outline of the scope and constraints under which we develop it. The chapter concludes an overview of the report's structure.

1.1 Background & Motivation

Companies continuously signal their strategic direction through observable digital activity – hiring patterns, product launches, partnerships and financial development – often long before these intentions are made explicit. Detecting these signals early offers a critical timing advantage in business development. However, identifying such signals at scale remains a significant challenge. Relevant information is fragmented across public sources, manual monitoring is inconsistent and slow, and most

signals go unnoticed or are detected by chance rather than through any systematic process (Mühlroth et al., 2023). Recent advances in large language models create a concrete opportunity to address this: automated collection and AI-supported interpretation can replace fragmented manual workflows with a scalable, repeatable approach. This motivates the development of the Business Development Radar presented in this report.

1.2 Problem Statement

The gap is especially pronounced in contexts where business development teams must rapidly assess multiple companies and tailor their service offerings to each company's specific current needs such as the Innovation Day event at the House of Communication, where teams present directly to clients and concrete, up-to-date company knowledge is essential.

The core challenge is not data availability since relevant company information is publicly accessible across press releases, job postings, financial reports, and product updates. The problem lies in analysing these signals on time and dicing meaningful context out of them. Signals vary considerably in strength and clarity: some are explicit and immediately actionable, while others are weak and indirect, only becoming meaningful, when interpreted in context or in combination with other observations (Ansoff, 1975; Hiltunen, 2008). Identifying and weighing these signals consistently across large volumes of unstructured data is beyond what manual workflows can reliably deliver.

Current approaches rely largely on individual analysts monitoring diverse sources using their own tools and judgement, which is a process that is time-consuming, difficult to scale, and prone to inconsistency. Computational methods and data analytics have the potential to identify patterns from publicly available structured and unstructured data, yet these approaches remain underutilized in business development practice. This project addresses the central problem as a lack of scalable, systematic method for detecting, classifying, and interpreting company signals. The Business Development Radar responds to this by combining automated data collection with AI-supported interpretation, providing a stronger basis for targeted, evidence-based business development decisions.

1.3 Objectives of the Project

This project pursues three objectives. The primary objective is to develop an automated Business Development Radar that collects and processes relevant company signals from publicly available web-based sources, replacing manual and fragmented monitoring with a structured, scalable pipeline.

The second objective is to reliably distinguish between noise, weak signals, and strong signals within large volumes of unstructured data, while prioritizing information based on strategic relevance and

identifying patterns across signals over time to surface broader trends that would remain invisible through isolated observations alone.

The third objective is to translate detected signals into structured insights and actionable recommendations that directly support business development decision-making. The final deliverable is a functional prototype demonstrating how signal detection, interpretation, and recommendation generation can be systematically automated, serving as a foundation for further organizational scaling.

1.4 Scope & Constraints

The Business Development Radar is scoped to support the business development activities of Plan.Net TechNest, focusing on the automated collection and interpretation of publicly available company signals from web-based sources. The system is designed as a functional prototype, meaning it demonstrates feasibility and analytical value within a defined organizational context rather than constituting a fully production-ready development. In the domain of Business Intelligence, prototypes are used to validate requirements, determine feasibility, and engage users for feedback, with iterative feedback loops considered essential to the development process (van der Lans, 2012). Accordingly, the current implementation targets a limited set of companies and sources, with the explicit intention to extend the scope in a subsequent development phase to cover the broader Plan.Net Group or Service Plan Group organization.

Several constraints define what the system deliberately does not do.

First, it does not aim to predict or forecast the financial performance of monitored companies. The system is rather designed to detect and interpret strategic signals, such as hiring patterns, partnerships, or product development. Second, the system does not automatically generate or dispatch commercial service offerings to monitored companies. All generated insights and recommendations are intended exclusively for internal use by business development teams, supporting human judgement rather than replacing it or triggering unsolicited external contact.

These constraints reflect a deliberate design choice to prioritize interpretive depth and analytical usefulness over automation breadth. We designed the prototype architecture with scalability in mind, so that future development can extend coverage to additional client segments, integrate new data sources, or embed the system more deeply into existing organizational workflows – building on the signal detection and interpretation framework that the current implementation establishes.

1.5 Structure of the Paper

This report is organized into eleven main chapters, followed by the references and appendix. Chapter 1 introduces the motivation, problem statement, objectives, and scope of the project. Chapter 2 provides the theoretical foundation for the Business Development Radar, focusing on web data extraction, data processing, legal and ethical considerations, and LLM-based business insight generation. Chapter 3 presents the legal assessment process we used to evaluate which sources can be included in the data pipeline.

Chapter 4 gives a high-level overview of the system architecture. Chapter 5 describes the data source selection and preparation pipeline, including scraping, API integration, database persistence, PDF processing, and data-related challenges. Chapter 6 explains the LLM-based signal classification component, while Chapter 7 presents the storage and retrieval logic we used to make classified signals available for later analysis. Chapter 8 describes the company-level reasoning layer, where the system aggregates into broader patterns and synthesizes them into actionable insights using a second LLM.

Chapter 9 presents the user interface and interaction concept, explaining how the prototype makes the system outputs accessible to business development users. Chapter 10 evaluates the system through qualitative assessment, error analysis, and limitations and concludes the report by summarizing the project contributions. Finally, Chapter 11 refers to individual contribution within the scope of the project study.

2. Literature Review

The development of the Business Development Radar draws on two interconnected bodies of literature: research on web data collection and processing, and work on AI-based interpretation of unstructured text. The following sections review the most relevant concepts, approaches, and challenges, moving from data acquisition through to signal classification and output generation.

2.1 Nature of Web Data and Its Challenges

The web offers organizations real-time access to vast amounts of publicly available information, like market signals, competitor activity, and emerging business trends (Ferrara et al., 2014). However, its practical value depends entirely on whether it can be collected and processed reliably. Web data presents two fundamental challenges: structural inconsistency and volatility. Web data lacks a consistent schema, arrives in multiple formats, and comes from sources that evolve continuously. Structural changes happen without forewarning, making any extraction system that depends on a fixed page structure fragile by design (Ferrara et al., 2014). Together, these mean that raw web data cannot be directly used

for analysis. A deliberate processing pipeline is required to transform it into a usable form, which is a challenge that shapes all the technical design choices we examine in the sections that follow.

2.2 Approaches to Extracting Value from Web Data

Given these challenges, the practical question becomes how to access web data at all. The two dominant approaches are APIs and web scraping. APIs provide structured access to backend data in a defined format, while scraping involves automated programs accessing websites and downloading their HTML content directly (Foerderer, 2023). The key asymmetry is reach: while APIs offer cleaner, more reliable data, the larger share of publicly relevant web content - competitor websites, news outlets, press portals and financial registries – is only accessible in semi-structured HTML form and exposes no API at all (Ferrara et al., 2014). APIs are subject to rate limits, scope restrictions, and provider decisions about what to expose, whereas scraping offers broader reach but introduces challenges related to volatile and inconsistent web content that can introduce sampling bias if not carefully managed (Foerderer, 2023). This asymmetry is the key reason why web scraping occupies a central place in our data collection.

2.3 Web Scraping as a Data Collection Method

Web scraping refers to the use of automated programs to access websites and extract their content into a structured format suitable for downstream analysis (Foerderer, 2023; Ferrara et al., 2014). Its core advantage over manual collection is scalability: we can collect data periodically and at scale with minimal human involvement (Ferrara et al., 2014). However, scraping requires careful design to be reliable in practice due to operational limitations such as structural dependency on page layout, platform-imposed request rate restrictions, and session management complexity (Tirado et al., 2016).

2.4 Types of Web Scraping Techniques

The appropriate scraping technique is not a design preference, but a constraint imposed by how each target site delivers its content. The literature broadly distinguishes three approaches. Static scraping retrieves plain HTML directly and extracts elements using parsing libraries or XPath expressions, which is effective when page content is fixed, but brittle when layouts change (Glez-Pena et al., 2014). Dynamic scraping is used when content is generated at runtime through JavaScript, necessitating a full browser environment to simulate user interactions and trigger page rendering (Ferrara et al., 2014). Recursive scraping, or web crawling, handles content distributed across multiple linked pages by following hyperlinks outward from an entry point (Tirado et al., 2016).

In practice, we combine all three approaches, since different sources may require different strategies.

2.5 Legal and Ethical Constraints of Web Data Collection

Automated web data collection is not only a technical task but also requires legal and ethical consideration. Relevant factors include whether the data is publicly accessible, whether it contains personal information, whether website terms of service restrict automated access, and whether technical access rules such as robots.txt indicate limitations. These constraints are particularly important in the European Union, where the processing of personal data is subject to GDPR requirements.

For the Business Development Radar, this means that source selection could not be based only on technical feasibility or business relevance. We also assessed each candidate source with regard to accessibility, data type, contractual restrictions, and responsible scraping practices. Chapter 3 therefore presents the resulting inclusion and exclusion decisions from a legal perspective.

2.6 Processing and Structuring Extracted Web Data

Collecting raw web data is only the first step in a longer pipeline. Once the content is scraped, it exists in a form that is largely unusable for analysis: HTML markup mixed with navigation elements, inconsistent formats across sources, and duplicate entries from overlapping scrapes. We therefore implemented a preprocessing stage to filter irrelevant content, clean text, and standardize formats before analysis can take place (Haans & Mertens, 2026). Poor data quality at this stage translates directly into unreliable downstream analysis – it is the processing layer, not the extraction layer, that ultimately determines whether a business intelligence system can function reliably (Intergrate.io, 2026). This processing challenge is what motivates the pipeline design described in Chapter 5 and sets the stage for the LLM-based interpretation discussed in the following section.

2.7 Large Language Models for Business Insight Generation

Once data is cleaned and structured, the challenge shifts from acquisition to interpretation. Large Language Models (LLMs) are well-suited to this task: trained on large volumes of natural language data, they can perform text classification, summarization, and information extraction across heterogeneous, text-heavy content (NYU Libraries, 2025). In business intelligence applications, LLMs are effective at analysing news articles, press releases, and public documents to track competitor activities and surface, actionable insights (Seker, 2024; Soumyals, 2025). For the Business Development Radar, LLMs serve as the interpretation layer that transforms cleaned web content into categorized, summarized, and prioritized signals. Chapter 6 describes the specific design of this layer. The following section addresses the prompt engineering approach required to make it function reliably within an automated pipeline.

2.8 Prompt Engineering for Consistent Output

Deploying an LLM within an automated pipeline introduces a challenge absent from interactive use: outputs must be not only analytically accurate but consistently structured and machine-readable for downstream processing. The effectiveness of LLM outputs depends substantially on how prompts are constructed. Prompt style significantly affects output quality, token cost, and processing time (Elnashar et al., 2025). For the Business Development Radar, we designed prompts to instruct the model to return outputs in a fixed JSON schema specifying fields such as topic category, relevance score, source type, and summary, so that the application layer could directly ingest every response. Techniques such as Chain-of-Thought prompting further improve performance on complex classification tasks by guiding the model through intermediate reasoning steps (Sahoo et al., 2024). Output formatting is a well-documented source of pipeline failures in production settings – a prompt that returns clean output on one model version may produce malformed output on another, causing downstream parsers to fail even when the underlying content is correct (Kotte, 2025). Mitigating this requires systematic prompt testing, validation logic, and where possible, enforcing output schemas through model-native structured output features.

3. Legal Assessment Process

The Business Development Radar relies on the automated collection of publicly accessible company information. Because web scraping can raise contractual, data protection, and ethical concerns, we assessed each candidate's source before inclusion in the data pipeline. The purpose of this assessment is not to provide a comprehensive legal opinion, but to define practical boundaries for the prototype: which sources we could use, which required constraints, and which we had to exclude.

We focused the assessment on four criteria: public accessibility, robots.txt and technical restrictions, Terms of Service or legal-notice restrictions, and whether the collected data was personal or company-level information. These criteria reflect the main legal and ethical dimensions commonly associated with web scraping, including contractual restrictions, data protection obligations, and responsible access practices (Brown et al., 2025; Krotov & Silva, 2018). Based on these criteria, we classified sources as low, medium, or high risk and either included them, included them with constraints, or excluded them.

3.1. Purpose and Assessment Criteria

A legal assessment is necessary because technical accessibility alone does not determine whether a source should be used. Even publicly visible information may be subject to contractual restrictions through Terms of Service, technical restrictions through robots.txt, or data protection requirements

where personal data is involved (Brown et al., 2025; Krotov & Silva, 2018). Because the radar focuses on internal business intelligence and on company-level signals, we aim to minimize legal exposure by collecting only publicly accessible, non-personal, business-relevant information.

The assessment therefore considers four criteria. First, the source has to be publicly accessible without login or technical circumvention. Second, we check robots.txt as an indicator of whether automated access was permitted or restricted, while recognizing that robots.txt is primarily a technical and ethical access signal rather than a standalone legal permission (Krotov & Silva, 2018). Third, we review the website's Terms of Service, legal notice, or impressum for explicit restrictions on automated collection or commercial use, since contractual restrictions can create legal exposure independently of whether data is publicly visible (Brown et al., 2025). Fourth, we assess the type of data to distinguish company-level information from personal data. This distinction is important because the GDPR defines personal data as information relating to an identified or identifiable natural person, while we primarily target organizational signals rather than individual profiles (Regulation (EU) 2016/679, Art. 4(1)).

3.2. Source Assessment Procedure

We evaluated each candidate source using the same six-step procedure. This ensures that source decisions are transparent, comparable, and documented consistently across companies and signal categories.

Step	Assessment Question	Decision Relevance
1. Identify Source	Does the source contain potentially relevant business development signals?	Determines whether the source is worth evaluating further.
2. Check robots.txt	Does the website indicate restrictions on automated access?	Provides a technical and ethical access signal for crawler use (Krotov & Silva, 2018).
3. Review legal notice / Terms of Service	Are there explicit restrictions on scraping, automated access, or commercial use?	Identifies contractual constraints that may apply independently of technical access (Brown et al., 2025).
4. Classify data type	Does the source contain company-level information or personal data?	Determines whether GDPR-related sensitivity is relevant (Regulation (EU) 2016/679, Art. 4(1)).
5. Assign risk level	Based on access, restrictions, and data type, is the source low, medium, or high risk?	Supports consistent inclusion or exclusion decisions.

6. Decide inclusion	Should the source be included, included with constraints, or excluded?	Defines the final source decision for the pipeline and documents the rationale.
---------------------	--	---

Table 3.1. Six-Step source assessment procedure

3.3. Risk Classification

We classify sources into three risk levels based on the combination of technical access, contractual restrictions, data type, and intended use within the prototype.

Risk Level	Criteria	Project Decision
Low risk	Publicly accessible source; no explicit scraping prohibition; company-level, non-personal information; internal analytical use.	Accepted as inputs, accessed with responsible identification
Medium risk	Ambiguous or partial restrictions; mixed company-level and incidental personal information; source useful but requiring careful limitation.	Testing with safe & responsible access methods required
High risk	Explicit scraping prohibition; predominantly personal data; login or authentication required; technical circumvention or bypassing required.	Rejected as inputs

Table 3.2. Risk classification of candidate data sources

This classification allows the team to make source decisions consistently while keeping the prototype within a clearly defined legal and ethical scope. The radar prioritizes public, non-personal, company-level information and excludes sources that would require authentication, circumvention, or collection of personal profile data. This approach aligns with the project’s internal business intelligence purpose and with the principle of limiting data collection to information necessary for the stated analytical objective (Bereczki & Liber, 2024; Infomineo, 2025).

3.4. Platform Restricted Sources based on the Example LinkedIn

We considered LinkedIn because it can provide valuable hiring, leadership, and professional activity signals. However, we classified it as high risk and excluded it from the automated data collection.

The main reason is contractual. LinkedIn explicitly restricts the use of crawlers, bots, scripts, browser extensions, or similar tools to scrape or copy profiles and other data from its services (LinkedIn, 2024). This restriction applies independently of whether some profile or company-page information is publicly visible. In addition, much of the relevant LinkedIn data relates to identifiable individuals, such as names, job titles, employment histories, and professional activity. This increases data protection sensitivity

because such information can fall within the GDPR definition of personal data (Regulation (EU) 2016/679, Art. 4(1)).

For these reasons, we excluded LinkedIn from the automated scraping pipeline. Instead, we rely on company-owned public sources, job pages, press pages, financial documents, and North Data, where the legal and technical access conditions are more compatible with our prototype scope.

3.5 Alternative Access Approaches

We considered alternative access approaches for LinkedIn-related signals, including third-party data providers and official API-based access. However, we did not adopt these options. Third-party providers do not remove the underlying contractual and data protection concerns, because the data would still originate from a restricted platform and can still involve personal profile information (LinkedIn, 2024; Regulation (EU) 2016/679, Art. 4(1)).

Official LinkedIn access routes, such as Sales Navigator integrations, are also not suitable for our purpose. These access routes are designed primarily for CRM-related workflows and do not provide the type of broad signal extraction required for the Business Development Radar. As a result, we did not implement a LinkedIn-based alternative. We treated the exclusion of LinkedIn as a deliberate scope decision rather than a technical limitation.

3.6 Technically Restricted Sources and Responsible Access

During source assessment (Section 5.1), we identified several candidate websites that were technically protected against automated access, most notably through Cloudflare or similar anti-bot mechanisms. These sources were potentially relevant from a business-development perspective, but standard crawling methods could not access them reliably, returning blocked responses.

Although bypass methods do exist, such as proxy-based access and rotating headers, we deliberately did not implement them, based on the same legal and ethical principles that guided the exclusion of other high-risk sources. The existence of such bot-protection methods means that website owners intentionally disallow bot access to their content. As bypassing these protections would mean deliberate self-misidentification, they conflict with our responsible scraping boundaries.

Where no suitable alternative source was available, we documented the resulting gap in the Source Map and Coverage Map (see details in Section 5.1).

3.7 Final Source Decision

The final source set is limited to publicly accessible sources that could be used without login, technical circumvention, or collection of personal profile data. Low-risk sources are included directly, while medium-risk sources are used only after testing responsible access. High-risk sources, including LinkedIn and sources requiring bypass techniques, are excluded from automated scraping.

All source decisions are documented in the Source Map and Coverage Map (see details in 5.1). This creates a transparent record of why each source was included or excluded and makes it possible to identify coverage gaps resulting from legal, technical, or accessibility constraints. The legal assessment therefore shapes not only the compliance boundaries of the prototype, but also the final source coverage available for downstream signal detection and analysis.

4. High-Level System Overview & Architecture

The Business Development Radar is structured as a modular pipeline that turns public company information into evidence-based business development insights. Rather than treating the system as one single AI step, the prototype separates the workflow into several connected layers: data acquisition, preprocessing and storage, signal classification, retrieval, company-level synthesis, and user-facing exploration.

At the beginning of the pipeline, the system collects information from different public and structured sources, including company websites, PDF reports, news feeds, job postings, and North Data outputs. Since these sources vary strongly in structure and quality, the collected material first needs to be cleaned, deduplicated, parsed, and segmented into units that can be processed reliably.

A key architectural decision is that raw and processed content is stored before LLM-based classification takes place. This means the system does not directly pass scraped content into a model and lose traceability. Instead, the database acts as a persistent foundation where collected content and later enrichments can be stored, reused, and inspected.

The first language-model stage, LLM1, operates on individual evidence units. Its task is to decide whether a piece of content contains a relevant business signal and, if so, classify it into a structured format. This includes fields such as category, bucket (signal strength), direction, confidence, evidence, summary, and business relevance. Signals classified as highly or weakly relevant are stored as enriched signal records, while irrelevant (noise) outputs are stored separately and excluded from later synthesis.

The retrieval layer then selects relevant stored signals for further analysis. It can use exact metadata filters as well as similarity search on previously generated vector embeddings. This layer forms the bridge between the classified signal database and the second language-model stage.

The second language-model stage, LLM2, works at a higher level. Instead of classifying individual pieces of evidence, it receives a retrieved set of previously structured signals and synthesizes them into company-level findings. This separation between LLM1 and LLM2 is central to the system design: LLM1 creates structured evidence, while LLM2 interprets patterns across that evidence.

The final layer is the user interface. It allows users to select companies and categories, retrieve evidence, run synthesis, and navigate between high-level takeaways, grouped findings, source-level evidence, and supporting company context. In this way, the interface makes the underlying pipeline usable for business development users without requiring them to understand the technical details of each backend step.

Figure 4.1 provides a visual overview of this architecture.

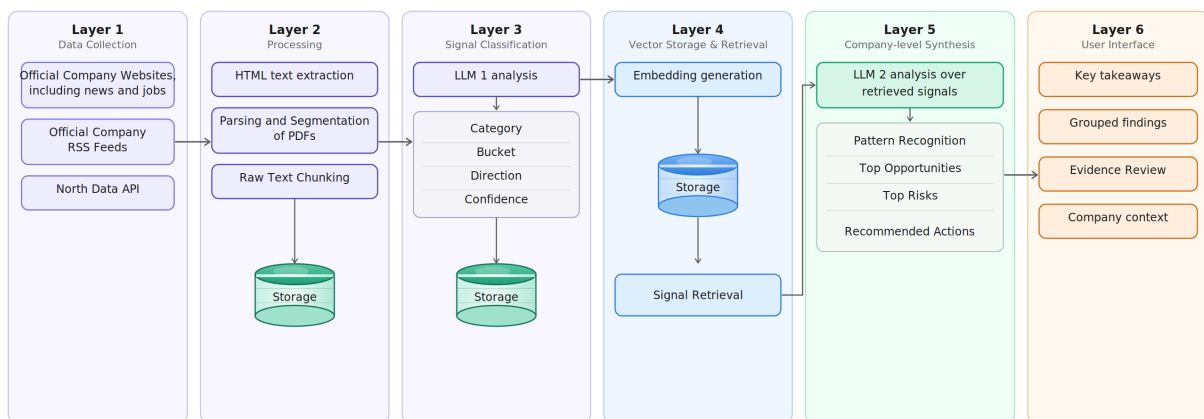


Figure 4.1 System Architecture Diagram

The figure shows how public sources enter the pipeline, how the system stores raw content before LLM1, how relevant and noise outputs are separated after classification, how storage and retrieval are split into distinct stages, and how LLM2 outputs are finally presented in the user interface.

5. Data Sources and Data Preparation

Building an automated Business Development Radar depends not only on the AI layer that interprets data, but equally on the quality and reliability of the data that feeds into it. Before any signal can be detected or classified, the right information must be collected from the right places, in a format that the system can process. This section documents how the system builds that foundation. It covers the selection and evaluation of data sources we use in the prototype, the technical strategies applied to collect content from company websites, the handling of PDF documents as a distinct data type, and the practical challenges encountered during data preparation. Together, these steps form the data pipeline that sits between the raw data sources and the LLM-based analysis described in the sections that follow.

5.1 Data Sources

The Business Development Radar draws on a deliberately selected set of public data sources across 21 monitored companies, spanning five source families: company websites, news and RSS feeds, job pages, financial documents and investor relations pages, and North Data registry outputs. We have chosen each of these source families to surface a different signal category: Products & News, Hiring, Financials, Partnerships & Acquisitions, Innovative Themes and Legal & C-Level Updates. Rather than scraping indiscriminately, we first defined the signal categories the system needed to cover and then identified which public source types are most likely to yield relevant information for each. This category-first approach reflects a broader principle in business intelligence system design, where source selection should be driven by analytical requirements rather than technical availability (Chaudhuri et al., 2011).

To make source discovery concrete and auditable, we produced two structured artifacts: a Source Map and a Coverage Map. Our work began from a list of 21 companies provided by Plan.Net TechNest. For each company, we manually explored its public web presence and identified candidate source entry points for each signal category. The Source Map recorded not only URLs but also evaluation metadata: average posting frequency, robots.txt and legal-imprint status, availability of RSS feeds and sitemaps, whether login was required, and an initial assessment of whether the source was genuinely useful for the intended category. This turned source discovery into a structured selection and validation exercise rather than a purely technical crawling decision.

Three criteria shaped our final selection. The first criterion was the category relevance: could the source plausibly surface one of the five signal types? The second criterion was the freshness: we used visible post-dates and archive observations to estimate whether the source updated frequently enough to support ongoing monitoring, since the value of a data source for competitive intelligence depends directly on its update frequency (Ferrara et al., 2014). The third criterion was the accessibility: whether

the source was technically reachable, whether RSS feeds or sitemaps simplified discovery, and whether robots.txt or terms of service indicated legal or technical restrictions. The Source Map explicitly tracked all these properties, including notes on when recursive crawling would be required, when access depended on login, and when scraping was restricted or disallowed.

The second artifact named Coverage Map complemented the Source Map by showing category coverage at company level. Where the Source Map was organized around individual URLs and their properties, the Coverage Map made category gaps visible at the level of each monitored company. This step revealed important structural patterns in source availability. Public financial information was limited or absent for several private companies (e.g., ECE Group, Ehrmann), where open financial coverage was only partial. In other cases, relevant information was accessible only through parent-company reporting structures (e.g., Penny via REWE Group, Smart via its parent company). These structural limitations are consistent with the wider observation in competitive intelligence research that private companies systematically disclose less information through public channels, requiring supplementary data strategies (Mühlroth et al., 2023).

The Coverage Map revealed three patterns that directly shaped downstream signal quality. Products & News and Hiring achieved the strongest coverage across the monitored set, with usable sources identified for nearly all 21 companies. Partnerships & Acquisitions and Innovative Themes were well-covered for larger public companies but varied depending on how prominently each company communicated these topics externally. Financials was the most uneven category: public companies (e.g., BMW, Lufthansa, Berlin Airport) had strong coverage through investor relations pages and annual reports, while private companies relied entirely on North Data as the only structured financial signal source. Additional constraints further narrowed coverage, for instance, Olympus's German career page and German news section were fully disallowed for scraping, forcing reliance on global sources for hiring signals while Cloudflare-protected sources across several companies could not be accessed without bypass techniques the project deliberately excluded on legal grounds. These gaps were documented in the Coverage Map and treated as known scope boundaries, making the limits of signal availability explicit for downstream analysis. Section 10.1 provides detailed tables on category coverage across companies.

Each source family served a distinct analytical purpose. News and press pages were the primary source for externally communicated company developments such as product launches, partnerships, milestones, and strategic announcements. Career and job pages were included because hiring activity can reveal capability build-up and internal priority shifts before they become visible in official announcements (Mühlroth et al., 2023). Financial documents and investor-relations sections provided formalized information on performance, investment direction, and capital developments, though these sources tended to be periodic and structurally more complex than standard web pages. Innovation and

strategy pages were relevant for companies that present digital transformation themes, R&D programs, or sustainability initiatives in dedicated sections outside their main news flow. Finally, we integrated North Data as a structured complement to company-owned sources. It aggregates publicly available registry data such as governance changes, ownership structures, financial filings, management appointments. North Data therefore provided access to a type of signal that companies would not voluntarily publish on their own websites and was particularly valuable for private companies where self-disclosed information was limited.

Overall, the combination of Source Map and Coverage Map ensured that source discovery was transparent, category-driven, and traceable. These artifacts are also useful from a scalability perspective, because they created a reusable structure for extending the prototype to additional companies and source sets without restarting source discovery from scratch each time. At the same time, the process made it clear that signal quality across the prototype is not uniform, but it depends directly on the availability, accessibility, and relevance of public sources for each individual company.

The following subsections explain how we accessed the identified sources technically, how we processed PDF documents as a distinct data type, and which practical challenges arose during data collection.

5.2 Scraping Strategy & Methods

To collect data from the public sources identified during the source selection process, we used web scraping to automatically gather web data at scale. This allows us to keep the dataset up to date with minimal manual effort, providing the foundation for later processing and LLM-based analysis.

We selected Scrapy as the main web scraping and crawling framework because it provides a comprehensive modular structure, with configurable crawling methods and support for large scale scraping (Wu, 2019; Chang, 2022). This made it suitable for collecting data from multiple public company websites while passing extracted items into later database-oriented processing steps such as filtering, deduplication, and storage.

Since the selected sources differ in structure and technical accessibility, we had to use multiple crawling methods. We chose the simplest reliable method for each source, reducing unnecessary complexity while still allowing the crawler to access the required content efficiently.

5.2.1 Structured Source Discovery: RSS Feeds and Sitemaps

We prioritized websites that expose content in structured formats, as they are the most efficient and reliable way to crawl webpages, since the crawler can identify relevant entries with a single step.

RSS feeds were especially useful, as they contain frequently updated website content such as press releases (RSS Advisory Board, 2009). We configured a Scrapy-based RSS feed spider to extract entries from the identified feeds, and extracted the full page content when it was accessible.

We used sitemap crawling as the second structured discovery method. Sitemaps list important website URLs and metadata, which helps crawlers discover relevant pages efficiently (Sitemaps.org, n.d.). We extended Scrapy’s SitemapSpider to fit the project structure and to extract URLs and metadata from sitemap files where available.

5.2.2 Unstructured Source Discovery: Recursion

Although structured sources were useful, many sources did not provide complete RSS feeds or sitemaps. Recursive crawling allowed starting from a defined page, extracting links, following them, and repeating the process until the crawler reached a stopping condition such as a set maximum depth or the absence of new relevant links.

This method highly increased coverage for less structured websites, but it also introduced more noise and higher crawling cost. For that reason, we treated recursive crawling as a fallback method rather than the default approach. We adjusted the maximum crawl depth for each source and used shallow recursion where a single link-following step was enough. A detailed recursion diagram is shown in Appendix A.

5.2.3 PDF-Oriented Crawling

We identified some sources that only include PDF documents as having all available content presented in a single webpage. Because only a single webpage needed to be accessed, the approach consisted of simply extracting links and metadata from identified HTML blocks, as explained in section 5.3.

5.2.4 Dynamic Source Access with Playwright

Some sources returned incomplete results when accessed through static Scrapy requests because relevant content was rendered through JavaScript. To be able to extract content from these pages, we integrated scrapy-playwright into the Scrapy framework. Playwright allowed us to load webpages in a headless browser and make JavaScript-generated content available for extraction (Microsoft, n.d.; Scrapy-Playwright contributors, n.d.). We chose Playwright specifically because of its seamless integration with Scrapy, and its relative simplicity.

We used Playwright selectively because it increases runtime and implementation complexity compared with static crawling. For some dynamic sources, rendering the page was enough to expose the required

content. For others, we added simple actions such as scrolling to ensure that all available content loaded before extraction.

5.2.5 Dynamic Source-Specific Page Interaction

For four edge-case sources, rendering the page alone was not sufficient. These sources required custom interactions, such as opening dropdown menus, loading hidden content, or following dynamic buttons that led to additional pages. We implemented source-specific Playwright interactions for these cases and documented the pseudocode in Appendix B.

This approach improved coverage for the most technically difficult sources, but it also introduced a maintenance risk. Because the interaction logic depends on each website’s structure, future website updates may break the crawler and require adjustments.

5.2.6 Filtering and Data Quality

We applied filtering during crawling to reduce irrelevant, outdated, or incomplete content before it entered the downstream pipeline. This was necessary because later stages, including storage, deduplication, LLM-based enrichment, and retrieval, depend on the quality of the collected input.

We implemented source-specific filtering through modular configuration files. These settings allowed us to exclude irrelevant pages like low level jobs, restrict collection to relevant website areas, and attach pre-crawl metadata such as the company name. To avoid the collection of outdated data, we filtered out any webpage content older than 1 year, and PDFs older than 2 years. This made sure the stage 1 LLM analysis wasn’t overloaded with a ton of irrelevant input.

5.2.7 Responsible Access and Identification

As outlined in Section 3, legally and ethically responsible access to websites to be scraped was a key requirement. We used Scrapy’s default settings to identify clearly as a bot, and to always automatically obey robots.txt guidelines. We used Playwright’s headless browsers only to load content, and not to impersonate a human user.

We used Scrapy with the responsible settings as the last step in testing viability of medium and low risk sources, as outlined in Section 3.3. While testing, we identified that some medium-risk sources were inaccessible, and thus excluded them from integration.

5.2.8 Summary of Configurations per Source

All in all, we were able to implement 67 sources in total, with total counts for each approach outlined in table 5.1.

Data Collection Method	Source Count
Recursive Crawling	20
Sitemap Crawling	18
Single Site PDF collection	11
Dynamic (Playwright)	9
RSS Feed Crawling	6
Dynamic Crawling with Item Interactions (Playwright)	4
Total	67

Table 5.1: Configured source counts per web scraping method

5.3 HTML Extraction

After we identified the relevant webpages and retrieved their corresponding raw HTML source code, we needed to extract the meaningful text from the HTML to pass it down for processing.

5.3.1 Trafilatura

When processing web data at a large scale, minimizing irrelevant information coming from raw HTML pages becomes a major concern. These typically contain large amounts of non-essential content such as navigation elements or advertisements (Ferrara et al., 2014). Because the project accessed a heterogenous set of company webpages we needed a way to reliably extract key information stemming from these websites without defining custom rules for each of them.

To address this, we used the Python library Trafilatura to extract the content immediately after scraping it. Trafilatura identifies main textual content in HTML pages and removes unnecessary elements while preserving relevant text and metadata (Barbaresi, 2021). Trafilatura proved to be suitable for the project because it performed robustly across heterogeneous sources with varying website structure and it extracted the content using heuristics rather than deterministic, site-specific extraction rules (Barbaresi, 2021).

We therefore utilised Trafilatura directly after the Scrapy retrieval process for most of the accessed webpages, eliminating the need for manually defining extraction rules for many sources.

5.3.2 CSS selectors

Although Trafilatura proved effective for the majority of sources, there remained several edge cases for which we had to define custom extraction rules. Most of these outliers were company webpages containing financial reports which were not embedded directly in the HTML but rather hidden under PDF download links. Furthermore, we encountered a small number of webpages for which Trafilatura didn't extract the content reliably. Finally, when performing dynamic crawling we needed to precisely identify page elements, we wanted the headless browser to interact with.

We addressed this issue by accessing content using CSS selectors. The integration proved to be seamless since Scrapy already offered functionality to target HTML content by specifying the value of the class attribute of the relevant element. This made the custom extraction logic fit naturally into our system without the need for major architectural changes.

By utilizing CSS selectors, we were able to extract information from the edge cases reliably and enrich the radar with many high-quality sources.

5.4 Database Persistence and Deduplication of Scraped Sources

Once Scrapy collects the data, each item moves through the pipeline and enters the database (Scrapy developers, n.d.). This persistence layer provides later processing stages with stable and reusable entries.

Webpage results occasionally contained identical content under different URLs, which meant that identical content was stored repeatedly. Later analysis stages could then treat such duplicated content as separate and equally important signals, falsely strengthening patterns. Therefore, we implemented 3 approaches to remove duplicates content before saving to the database.

To reduce duplicates caused by minor URL variations, a URL normalization script cleans raw URLs by removing markers such as language parameters. To detect exact duplicate content efficiently, the system uses hashing, which converts each webpage's text into a short fingerprint, increasing the efficiency of duplicate checks over the entire database (Manning et al., 2008).

To identify near-duplicate webpages, the system uses MinHash Locality Sensitive Hashing (LSH). MinHash LSH represents webpage text as shingles, estimates similarity through MinHash signatures, and uses locality-sensitive hashing to compare only likely duplicate candidates in detail, enabling large-scale deduplication (Broder, 1997; Leskovec et al., 2020). Appendix C explains the MinHash LSH logic in detail.

New insertions therefore go through URL-level, exact-content, and fuzzy-duplicate checks before the system skips them, saves them, or updates an existing record.

5.5 API calls to North Data

North Data GmbH is a company that aggregates publicly available company information coming from sources such as trade registers, annual reports, patent registries, trademark records, and other official databases and provides access to it in a structured manner (North Data, n.d.). We decided that including such a source into our data pool would improve the quality of the outputted results. We also believed that it would fill information gaps for companies for which we could not extract some signal categories due to either legal, ethical or practical constraints such as the data not being made accessible via official company websites.

After obtaining an academic license for the platform we created a system that queries the North Data API on a weekly basis. We decided to fetch data such as company-related information, including financial history, key performance indicators over time, relationships to associated companies and individuals, and balance-sheet data from recent reporting periods. In addition, we also queried information about management changes, trademark and patent filings, mergers and acquisitions, and other corporate events relevant to our use case.

We implemented the data retrieval process by using Scrapy spiders directed toward the relevant API endpoints. This proved to be a superior solution as Scrapy was already the backbone library of the project and there were no extra architectural changes needed to be made in order to integrate North Data information into the pipeline using the existing approach.

Including North Data into our system ultimately provided valuable insight into recent company developments and enriched the radar's results. Furthermore, the key performance indicators, as well as company relationships were incorporated into the final UI to give an overview of company health and structure.

5.6 PDF Processing

PDF documents, particularly financial reports, were a key information source for the Business Development Radar. These typically contained indicators such as financial performance over time, key metrics and strategic outlooks. Because these documents contained such valuable business insights, we needed to create a processing pipeline which would extract this relevant information in a reliable and consistent manner.

We incorporated IBM's Docling library as the primary framework at the PDF parsing stage. It proved to be useful for several reasons. Firstly, its models are noted for strong accuracy and computational efficiency. Furthermore, Docling supports the extraction of embedded images and can generate natural-language descriptions of charts and other figures (Livathinos et al., 2024).

We used OpenAI’s GPT-4.1 mini model to describe the images. Images detected by Docling were sent to the model to be processed via API calls with the following prompt: “Describe the image in three sentences. Be concise and accurate”. The generated descriptions proved to be sufficiently detailed for the the projects use case while remaining computationally practical for large-scale document processing.

The results of the parsing stage can be seen in figure 5.1.

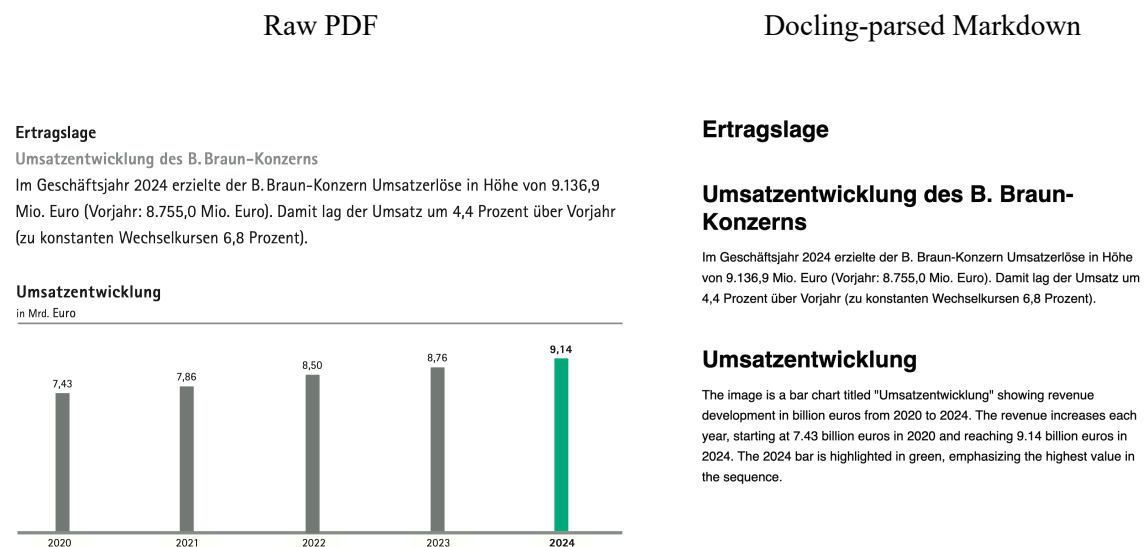


Figure 5.1. Financial report and corresponding Docling-parsed markdown

After parsing, the extracted documents were divided into smaller sections referred to as chunks.

Chunking was necessary because vector embeddings could only be generated up to a limited size of tokens. We limited chunk size to 512 tokens, as it was the maximum limit for the embedding model that we used, this is further explained in the vector embeddings section. Docling’s hybrid chunking mechanism was particularly useful because it preserved document structure while generating chunks. Rather than splitting text arbitrarily, the chunking process attempted to preserve headings, tables, and contextual relationships between sections (Docling Project, n.d.). We expected this to improve the quality of vector retrieval. Finally, the generated chunks were embedded and stored in a vector database together with their raw text representations.

Larger segments of the PDF were also needed to analyse the context of larger semantic sections of the PDF. For this, all chunks that share the same headings (extracted by Docling) were combined to form segments. Adjacent small segments that could have formed a combined segment of a maximum of 500 tokens were merged. The resulting segments were stored and used for stage 1 LLM analysis.

6. LLM-Based Signal Classification

Once we have collected and cleaned content from the various public sources, the practical question became: which of these text fragments actually say something worth paying attention to? Even after scraping and normalization, the data is still a mixture. Some of it is genuinely informative, some of it is peripheral background, and a large portion is simply noise. Separating these is not a trivial task, and keyword matching alone cannot do it reliably. This section describes how we approached that problem through a first-stage language model layer, referred to throughout as LLM1, and explains the design decisions behind it.

6.1 Rationale for Using Large Language Models

The core difficulty in classifying signals from web-scraped content is that the same type of development, such as a leadership change or a new technology investment, can appear in very different forms depending on the source. A press release will state it directly while a financial report might imply it through budget figures. A job listing might hint at it through the roles being advertised. Rule-based systems or keyword filters can catch obvious cases but break down quickly when the strategic implication depends on context rather than specific wording. As Adnan and Akbar (2019) note, unstructured web data is by nature noisy and inconsistent, which makes fixed extraction logic fragile across diverse source types.

We selected GPT-4.1 mini for LLM1, as it offers strong contextual reasoning at a lower cost than larger models but also conceptually appropriate. The task here is not generating text or summarizing documents in an open-ended way. It is making a bounded interpretive judgement: does this fragment represent a business-relevant development for Plan.Net TechNest, and if so, what kind? That is exactly the type of task where language models have shown strong performance, particularly in business-oriented classification and information extraction from unstructured inputs (HCI International, 2023). Importantly, LLM1 does not produce final company-level assessments, that is handled later by a second model referred to as LLM2 (Section 8). Its role is narrower: evaluate each evidence unit individually and produce a structured output that can be stored and retrieved.

This modular design was a deliberate choice. Having one model attempt to classify individual fragments and reason about company-level strategy simultaneously would make both tasks harder to control and evaluate. Keeping LLM1 focused on a single, well-scoped classification problem made the outputs more consistent and the prompt easier to refine iteratively.

To make the output concrete, consider a job posting for a Head of Global SAP S/4HANA Program PMO at Olympus. LLM1 classified this as a main-bucket, high-confidence opportunity signal with

Hiring as the primary category and Innovative Themes as a secondary category. The evidence field captured the core content of the posting, including global ERP transformation, PMO governance and system integrator coordination. The short_summary field stated the strategic implication while the possible_business_suggestion field identified a concrete opening for PNTN in SAP implementation support and change management. This single structured record, inserted directly into the source_enrichments table, required no post-processing and was immediately available for retrieval and synthesis. Figure 6.1 in Section 6.2 shows the full classification and enrichment flow that produced it.

6.2 Signal Taxonomy and Classification Dimensions

By the time content reaches LLM1, it has already been scraped, cleaned, deduplicated chunked into segments of up to 512 tokens, and stored in the database. LLM1 operates on these stored chunks and its task is to decide whether each one carries a business-relevant signal and, if so, attach structured metadata to it. The output of this stage that is stored in the source_enrichments table is what all downstream retrieval, synthesis, and interface display builds on.

For LLM1 outputs to be useful for retrieval, filtering, and later synthesis, they needed to carry more information than just a topic label. We therefore designed the classification schema around several dimensions simultaneously.

The primary assignment is a signal category. The categories used in the project were: Financials, News / Products, Partnerships / Acquisitions, Hiring, Innovative Themes, and Legal & C-Level Updates. These reflect the signal types that Plan.Net TechNest identified as relevant to its business development context. Each signal can also be assigned secondary categories where a development genuinely spans more than one area. A senior hire in a newly created AI department, for example, is relevant to both Hiring and Innovative Themes. Forcing such cases into a single label would lose information that later stages could use.

Beyond category, through LLM1, each signal is assigned to one of three buckets: main, weak, or noise. These bucket labels are internal system categories used to distinguish explicit signals, early indicators, and irrelevant content for downstream retrieval and synthesis. This distinction matters for a specific reason. Early strategic signals are often incomplete or indirect. A single job posting does not confirm a major capability shift, but it might be the first visible indicator of one (Ansoff, 1975; Hiltunen, 2008). Discarding everything that falls short of explicit confirmation would lose exactly the material that an anticipatory intelligence system is supposed to surface (Lesca & Lesca, 2011). The three-bucket structure allows the system to preserve uncertain but potentially relevant evidence as weak-bucket signals rather than collapsing the decision into relevant or irrelevant. Retaining weak signals is especially important for the second reasoning stage because LLM2 (Section 8) can combine several

individually uncertain observations into a broader company-level pattern that would not be visible from any single signal alone.

The direction field that consists of opportunity, risk and neutral forms the third classification dimension. This is an early interpretive layer rather than a strategic recommendation. A signal about a company reducing its technology budget might be tagged as risk from a business development perspective; a new innovation initiative might be tagged as opportunity. The purpose is to give users and the retrieval layer something to filter on beyond category alone.

Finally, each output carries a confidence field: low, medium, or high. Confidence here reflects how clearly the available text supports the classification, not whether the underlying event is true. Short fragments, boilerplate passages, or partially extracted PDF sections tend to produce lower confidence even when categorized. Including this field makes the quality of each classification visible to downstream stages and to users inspecting the evidence view. Figure 6.1 summarizes the LLM1 classification and enrichment flow.

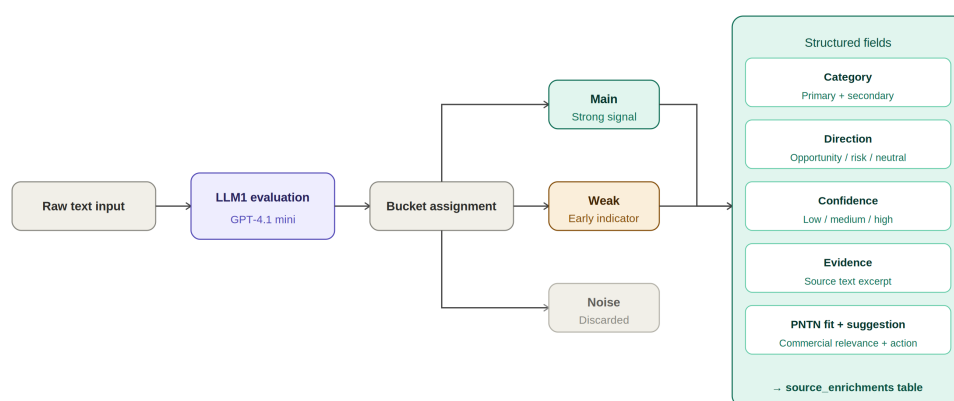


Figure 6.1: LLM1 signal classification and enrichment flow

Figure 6.1 shows the split that LLM1 creates after evaluating each evidence unit. Main and weak signals continue into the enriched signal path, where they receive category, direction, confidence, evidence, summary, and business relevance fields. Noise outputs are stored separately and are excluded from later synthesis. This distinction is important because it keeps potentially useful yet weak signals available for LLM2 while preventing irrelevant content from overloading the company-level reasoning stage.

6.3 Prompt Design and Output Structure

We designed LLM1 to return a fixed JSON object for every processed evidence unit. The schema defined three possible output formats depending on the bucket. A main-signal output contained the fields: `is_relevant`, `bucket`, `category`, `secondary_categories`, `signal_strength`, `pntn_fit_check`, `direction`, `evidence`, `short_summary`, `why_it_matters_for_pntn`, `possible_business_suggestion`, and `confidence`.

Weak-signal outputs shared the same fields but with `signal_strength` fixed at medium. Noise outputs returned only `is_relevant: false`, a result code, and a brief reason. Every LLM1 output could therefore be inserted directly into the database without any format negotiation or post-processing. We inserted relevant outputs into the `source_enrichments` table and noise outputs into the `source_enrichments_noise` table, keeping them separate due to their structural difference.

Table 6.1 summarizes the full output schema and all allowed field values for each structured dimension.

Field	Purpose	Allowed Values
<code>is_relevant</code>	true, false	Whether the chunk contains any signal
<code>bucket</code>	main, weak, noise	Signal strength tier
<code>category</code>	Financials, News / Products, Partnerships / Acquisitions, Hiring, Innovative Themes, Legal & C-Level Updates	Primary signal topic
<code>secondary_categories</code>	Same list as category	Cross-category signals where a development spans more than one area
<code>signal_strength</code>	strong (main signals), medium (weak signals)	Fixed per bucket, not model-assigned
<code>pntn_fit_check</code>	yes, no	Whether the signal represents a concrete delivery surface relevant to Plan.Net TechNest
<code>direction</code>	opportunity, risk, neutral	Business development framing of the signal
<code>evidence</code>	Free text (excerpt)	Exact text from source that supports the classification
<code>short_summary</code>	Free text	One-sentence strategic implication of the signal
<code>why_it_matters_for_pntn</code>	Free text	Explanation of commercial relevance to Plan.Net TechNest
<code>possible_business_suggestion</code>	Free text	Specific observation for the business development team
<code>confidence</code>	low, medium, high	How clearly the source text supports the classification

Table 6.1: LLM1 output schema and allowed field values.

The decision to enforce strict JSON rather than allow free-form responses was not just a technical convenience. Sahoo et al. (2024) show that explicitly constraining output format in the prompt significantly reduces variance across runs and prevents outputs from drifting in structure between similar inputs. In a pipeline where LLM1 outputs feed directly into database insertion and later downstream into the interface, even minor inconsistencies in field naming or structure would cause failures downstream.

We built the prompt around three points. First, a clear framing of the model's role: not a generic assistant, but a business development analyst evaluating content specifically for Plan.Net TechNest. This framing matters because relevance in this context is not general relevance. A story about a company opening a new warehouse might be noise for a technology consulting firm even though it is factually significant. Second, we spelled out the taxonomy explicitly: categories, bucket definitions, directional labels and confidence levels were all defined so the model would not invent its own logic. Third, we stated the output schema precisely, including the exact field names and allowed values for each. Elnashar et al. (2025) demonstrate that this kind of schema-level specification in the prompt substantially improves both accuracy and token efficiency in structured data generation tasks.

The `pntn_fit_check` field is worth noting separately. It required the model to reason explicitly about whether the signal was relevant to Plan.Net TechNest's service areas. This filtering step went beyond generic topic classification. It helped suppress signals that are technically interesting but commercially irrelevant. The output of this field is made accessible to user in the interface, described in Section 9, so users can see the reasoning behind why something was flagged.

A related design decision concerned how much project knowledge to hard code at company level. Instead of building separate explanations or prompt logic for each individual company, we worked with more general industry-oriented mappings and reusable interpretation patterns wherever possible. This reduced dependence on company-specific manual customization and made the classification logic easier to extend to new companies in later iterations. The prompt design was therefore shaped not only by accuracy requirements, but also by the need to keep the system scalable beyond the initial prototype scope. This was important because the prototype was intended to grow beyond the initial monitored company set without requiring company-specific prompt redesign for every new case.

One practical complication was source heterogeneity. The same prompt had to handle a paragraph from a German annual report, a two-sentence job listing, and a North Data publication entry. These differ not just in language and length but in what kind of signal they can realistically carry. We wrote the prompt to be tolerant of this variation at the input level while remaining strict at the output level.

6.4 Ambiguity, Edge Cases and Consistency Measures

Ambiguity in classification was a persistent challenge throughout development. Some fragments contained genuinely ambiguous content such as background company descriptions, partially extracted PDF sections, navigation text that Trafilatura did not fully clean, or passages that were relevant but only in combination with other signals not visible in that chunk. These cases cannot be resolved cleanly at the preprocessing stage (Ferrara et al., 2014).

We specifically designed the three-bucket system for this. Rather than forcing a binary decision, uncertain material could be routed to weak rather than noise, preserving it for later stages without giving it the same weight as explicit signals. This is conceptually consistent with how weak-signal methodology treats early strategic information: fragmentary observations that cannot be confirmed individually may still be worth retaining because they can gain meaning in combination with other evidence (Lesca & Lesca, 2011).

Consistency across runs was a separate problem. Because the same piece of text might be processed multiple times across different scraping cycles, the classification needed to be stable. We addressed this primarily through prompt specificity: enumerating all allowed field values, fixing the label wordings, and constraining free-form fields to specific purposes. When iterative testing showed that certain borderline cases were being classified inconsistently, we made the prompt description for the relevant categories more precise. Some residual inconsistency is unavoidable on genuinely ambiguous inputs, but the structured output format ensured that whatever classification was assigned was at the minimum machine-readable and retrievable.

6.5 Cost Awareness and Scalability Considerations

Running a language model on every scraped evidence unit across 21 companies, multiple source types, and weekly update cycles scales quickly. We chose GPT-4.1 mini partly because it handles this classification task well at a fraction of the cost of larger models, but model selection alone was not sufficient. Several upstream decisions also contributed to keeping inference volume manageable. More broadly, we did not design LLM1 as a one-off classification layer for the initial company set, but as a reusable intermediate structure that can support future expansion to additional companies, categories, and recurring update cycles without requiring a redesign of the whole interpretation logic.

Source-level and preprocessing-level filters reduced the number of fragments that reach LLM1 in the first place. We excluded job postings below a defined seniority threshold, filtered out articles older than one year and ensured duplicate content detected through the deduplication pipeline did not re-enter the classification queue. Storing LLM1 outputs in the database meant that already-classified signals could be retrieved in later sessions without re-running inference on unchanged content, which was an important consideration for a system designed to run on a recurring basis.

The separation between LLM1 and LLM2 (Section 8) also served a cost function. The lighter model handles the high-volume task of classifying individual evidence units, while the more capable and more expensive model is reserved for company-level synthesis, which happens at a much lower volume. This division made it practical to run the classification stage at realistic scale. Vertsel and Rumiantsev (2024)

make a similar argument for modular LLM architectures in business analytics: separating tasks by complexity allows each component to be optimized independently for both quality and cost.

7. Vector Storage and Retrieval

Collecting and classifying signals only becomes valuable when we can store, organize, and retrieve the results in a way that supports meaningful analysis. Once the first LLM layer has processed and classified a large body of collected data, the system needs a reliable way to surface relevant information on demand for later synthesis. We therefore implemented a data retrieval layer, to enable the system to fetch the most relevant stored entries.

7.1 Structured Metadata Retrieval

We implemented structured metadata retrieval as the default method for use cases where existing fields could clearly identify relevant entries. This method used SQL filters to fetch LLM1-enriched source entries based on metadata such as company name, category, and signal direction.

This approach was simple, transparent, and efficient when the user knew which filters to apply. However, it depended on predefined fields and could not identify related entries when they differed in wording, category, or metadata.

7.2 Vector Search

We implemented vector search because simply retrieving all results is not sufficient for large-scale, multi-company analysis, while exact metadata filtering alone is too limited. Large result sets can overwhelm downstream analysis, and relevant signals may not always share the same predefined categories or metadata fields. Vector search allows the system to retrieve semantically similar entries across raw text chunks and LLM-generated enrichment fields (Reimers & Gurevych, 2019). This allows broader analytical queries such as identifying specific business opportunities or strategic changes.

The retrieval system supports configurable filters over all existing fields, multiple query inputs, and similarity thresholds & size limits to keep results focused. After retrieval, it consolidates results at the enriched-source level, ranks them by similarity, adds relevant metadata, and formats them for the second LLM layer.

7.3 Vector Embedding Generation and Storage

To enable vector search over the database, we had to generate vector embeddings of fields that needed to be searchable. We used OpenAI’s text-embedding-3-large model because it provides strong semantic similarity performance and supports multilingual text (OpenAI, n.d.). We embedded the four non-discrete LLM1 outputs separately along with a combined block to support broader retrieval.

For raw text, we split entries into chunks of up to 500 tokens with a 50-token overlap to reduce the loss of meaning across chunk boundaries. The upstream PDF pipeline already generated PDF chunks. We stored the resulting embeddings in PostgreSQL using the pgvector extension, which allows semantic search to remain in the same relational database as the rest of the processed data and avoids the need for a separate vector database (pgvector, n.d.).

We saved chunk embeddings and enrichment embeddings in separate tables to support controlled retrieval across different content types and enrichment fields.

8. Pattern Analysis & Company-Level Reasoning

By the time signals reach this stage (LLM2), LLM1 has already processed each individual evidence chunk and produced a structured JSON record containing a category, direction, confidence level, evidence excerpt, short summary, and a Plan.Net TechNest relevance assessment. These records sit in the source_enrichments table and form the input to the second reasoning stage. LLM1 does not interpret patterns across signals or synthesize a coherent company-level picture, instead each record exists independently. The second reasoning stage, referred to as LLM2, addresses this gap. Its role is to receive a retrieved set of LLM1 outputs for a given company and scope and synthesize them into an actionable company-level assessment.

8.1 Why Isolated Chunk-Level Signals Are Not Sufficient

The limitation becomes clear with a concrete example. Suppose LLM1 has classified, for a given company over recent weeks, a strong Financials signal indicating increased R&D spend, a medium confidence Hiring signal around data engineering roles, and a weak Innovative Themes signal from an update to the company’s technology page. None of these are actionable individually. Together, they suggest a company that may be building toward a significant data or AI capability investment which is exactly the kind of emerging pattern that would be worth raising in a business development conversation. Ansoff (1975) made this point about strategic signals decades ago: their significance often only becomes visible when multiple weak observations are read together rather than in isolation. More recently, Lesca

and Lesca (2011) describe this as the core challenge of anticipatory intelligence assembling fragmentary observations into patterns that can support early action.

There is also a practical redundancy problem. The same company development signal frequently surfaces across multiple sources. Consider a partnership announcement that appears in a press release, a news article summary and indirectly in a financial report. Even after deduplication removes literal duplicates at the database level, semantically related signals can remain distributed across several retrieved records. Without an aggregation step, a user browsing the interface (see Section 9 for details) sees a fragmented list that requires significant manual interpretation to use. The purpose of LLM2 is to handle that interpretation systematically.

8.2 Company-Level Aggregation of Retrieved Signals

Before LLM2 can reason across signals, we assemble the relevant LLM1 outputs into a coherent evidence set. This happens through a retrieval step that selects signals from the database by company, category scope, time range, and other metadata attributes. The result is a bounded set of already-structured records which is not raw scraped text, but the classified intermediate outputs from LLM1 complete with their category, direction, confidence, evidence text, short summary, and the `pntn_fit_check` reasoning. LLM2 receives this set as its input.

This design matters for a specific reason. Giving LLM2 access to raw scraped content would require it to re-do the interpretation work that LLM1 already handled and would re-expose it to the full noise of the source material. By working from LLM1 outputs instead, LLM2 can focus entirely on reasoning across signals by identifying patterns, resolving tensions between contradictory signals, and synthesizing the collective picture. Badmus et al. (2024) describe a similar principle in AI-driven business analytics: structured preprocessing that standardizes evidence before higher-level reasoning is applied produces more reliable and interpretable outputs than end-to-end pipelines that attempt both tasks simultaneously.

The retrieval is also scope flexible. A user might request signals specifically for Hiring and Innovative Themes if they are getting prepared for a technology-focused client discussion or might pull across all categories for a broader company overview. LLM2 synthesizes whatever evidence set the retrieval returns, which means the scope of the final output is controlled by the user's filtering choices or query rather than fixed in advance. Because all signals remain database-backed and individually accessible, the retrieval layer can be adjusted without changing the synthesis logic. This separation also improves scalability, because company scope, category scope, and retrieval logic can be extended independently while keeping the second-stage synthesis step structurally stable.

8.3 LLM2-Based Synthesis and Final Output Generation

We implemented LLM2 using GPT-5.4, a more developed model rather than the mini variant used for LLM1. The reasoning demands of company-level synthesis are qualitatively different from per-unit classification. The model needs to recognize thematic patterns across multiple signals, weigh their collective direction, and produce output that a non-technical user can act on. That warrants a more capable model even at the cost of higher per-call pricing, given that synthesis runs at much lower volume than LLM1 classification.

The output structure of LLM2 reflects its role and feeds directly into the Key Takeaways and Findings views of the tool interface (Section 9). For each company and scope, LLM2 produces: an executive summary of the current signal landscape, an overall strategic direction and confidence estimate, grouped findings that identify thematic clusters across multiple signals, a prioritized set of top opportunities and emerging opportunities, a prioritized set of risks and recommended follow-up actions.

The grouped findings are analytically the most important part of the output. Rather than listing what individual signals said, LLM2 identifies which signals are saying related things and merges them into a broader observation. This means several separate signals about management reshuffling, a new technology hire, and a product page update might become a single finding about an organizational shift underway, which is something none of the individual signals could have supported alone. This is the core of what Lesca and Lesca (2011) mean by collective interpretation of weak signals.

To make this concrete, consider an LLM2 output for Scalable Capital across News / Products, Financials, Partnerships, and Hiring. From nine LLM1 signals, LLM2 produced three grouped findings. The primary finding was around Scalable Capital's transformation into a vertically integrated investment and banking platform backed by a €155 million funding round. The second finding was around new customer-facing digital products including an AI-powered knowledge base, kids' accounts, and savings accounts. The final finding was around strategic hiring in financial content roles. Using these findings, LLM2 identified top opportunities, emerging opportunities, and top risks alongside specific follow-up actions for the business development team. An example of how this output renders in the interface is shown in Figures 4 and 5 in Section 9.

The synthesis is framed explicitly as decision support rather than a definitive verdict. The outputs help a business development manager orient themselves before a meeting, not replace their judgement about whether and how to engage. This is also why the interface, described in Section 9, preserves access to the underlying LLM1 records so users can trace any finding back to its source signals. Şeker (2024) identifies this kind of source-grounded transparency as a core requirement for LLM tools used in business decision-making contexts, where credibility depends on the ability to validate outputs rather than trust them blindly.

8.4 Remaining Challenges of Company-Level Reasoning

The most direct limitation of the current LLM2 stage is its dependence on what enters it. If important signals are absent from the retrieved set for various reasons such as a source not being scraped, LLM1 under-classifying a relevant fragment, or the retrieval scope being too narrow, the synthesis will reflect those gaps without flagging them. LLM2 has no mechanism to know what it is missing. This means the quality of the final company-level output is genuinely upstream-dependent, and improving it requires improving the pipeline stages that precede it, not just the synthesis model.

Abstraction also introduces risk. The more LLM2 synthesizes across signals, the further its output moves from the source material. There is a real possibility of overgeneralizing from a thin evidence base, or of the model constructing a plausible-sounding narrative that is not well-supported by what was actually retrieved. This is partially mitigated by the interface design discussed in Section 9, which preserves access to the underlying LLM1 records, so users can trace any synthesis back to its source signals. In practice, however, busy business development teams are unlikely to inspect the evidence layer before every meeting which means the synthesis needs to be both useful and appropriately hedged about its own confidence.

A third challenge is context-sensitivity. Company-level reasoning is not context-free: the same cluster of signals can mean different things depending on the company's sector, its current relationship with Plan.Net TechNest, and what specific question the user is investigating. The current LLM2 prompt does not incorporate that kind of contextual framing. Future iterations could address this by-passing additional company background or user-specified context into the synthesis prompt, making the output more precisely calibrated to the specific situation rather than based only on what the signals say in isolation.

9. User Interface and Interaction Concept

The technical pipeline described in the preceding sections only creates value if its outputs are accessible to the people who need to use them. For the Business Development Radar, those people are business development managers and consultants at Plan.Net TechNest who need to quickly understand what is happening at a company before a client conversation or an internal prioritisation decision. We built the Streamlit-based interface around two user types and three design principles.

The first user type to use Business Development Radar is the business development manager preparing for a client-facing event. This user wants a fast, readable overview of recent company signals, a sense of whether the picture looks like an opportunity or a risk, and a pointer to the areas worth closer attention. Speed and interpretability matter more than technical depth, and the Key Takeaways tab serves as their

natural entry point. The second user type wants to verify the system's outputs before acting on them. After seeing a grouped finding, this user needs to trace it back to the signals that generated it and judge whether the evidence is convincing. Şeker (2024) notes that trust in AI-produced business intelligence is closely tied to the ability to inspect and validate the reasoning behind it, rather than accepting summaries at face value. Rather than building separate interfaces for these two user types, we handle both through a layered structure: high-level synthesis is always visible first, and deeper evidence layers are accessible to anyone who wants to go further.

Three principles shaped our interface throughout development. The first was layered access: the most synthesised output, which is the executive summary and opportunity/risk focus sections, is always what the user sees first. The grouped findings and individual evidence records are accessible through further navigation but never forced upfront. The second principle was business-facing language. Labels, section headings, and descriptions answer practical business development questions instead of describing the technical pipeline. As a result, technical terminology does not dominate the interface. The third principle was evidence visibility: the interface preserves access to the grouped findings and the underlying LLM1 records on the Evidence tab so users can check the basis for any claim the system makes, providing the transparency necessary for critical engagement rather than passive acceptance. The interface is organised as a six-tab application, Section 9.2 describes the structure and content of each tab in detail.

9.1 Core User Flows

The main flow through the interface starts with selecting one or more companies from the monitored set and choosing the signal categories to explore. The interaction sequence is straightforward:

1. Select one or more companies and relevant signal categories in the sidebar.
2. Retrieve the classified signals stored in the database.
3. Trigger the company-level synthesis based on the selected scope.
4. Explore the results across three analytical layers:
 - Key Takeaways for a high-level overview,
 - Findings for grouped interpretation,
 - Evidence for detailed signal verification.

This structure allows users to move from a quick business overview to deeper validation without needing technical knowledge of the underlying pipeline.

The system retrieves the relevant LLM1 signals from the database and runs the LLM2 synthesis over them, then displays the result. A consultant preparing for a meeting about a company's technology strategy might select Hiring and Innovative Themes; someone doing a broader company assessment might pull all categories. The synthesis adjusts to whatever scope was requested.

Direction-based filtering is another way of navigating the outputs. Users can focus specifically on opportunity-tagged signals, risk-tagged signals, or neutral developments depending on what question they are trying to answer. Because we assigned direction at the LLM1 level and preserved it through to the LLM2 grouped findings, filtering by direction shapes the interpretation presented to the user, not just the raw list of records.

The third flow is evidence inspection. After reading a grouped finding, a user can move to the Evidence tab to see the individual LLM1 records that support it. Each record shows the signal category, direction, confidence level, and the `pntn_fit_check` reasoning explaining why the signal was considered relevant to Plan.Net TechNest. This path through the interface is important for maintaining trust in the system's outputs, particularly for consequential decisions.

9.2 Prototype Structure and Screens

The prototype is organized as a six-tab Streamlit application. The tab structure mirrors the analytical layering of the system itself: high-level synthesis is presented first, grouped interpretation second, source-level evidence third, and structured company context in separate supporting views. The tabs correspond to distinct layers of the analytical pipeline and serve different user needs.

The Key Takeaways tab is the entry point and the layer most users will spend the most time in. It shows the LLM2-generated executive summary, an overall business development outlook, and three compact focus sections organized by direction: opportunities, emerging opportunities, and risks. The intention is that a user can read this tab in a few minutes and come away with a clear sense of where the company currently stands and where to focus further attention. Figures 9.1 and 9.2 show the Key Takeaways tab for Raiffeisen with all signal categories selected.

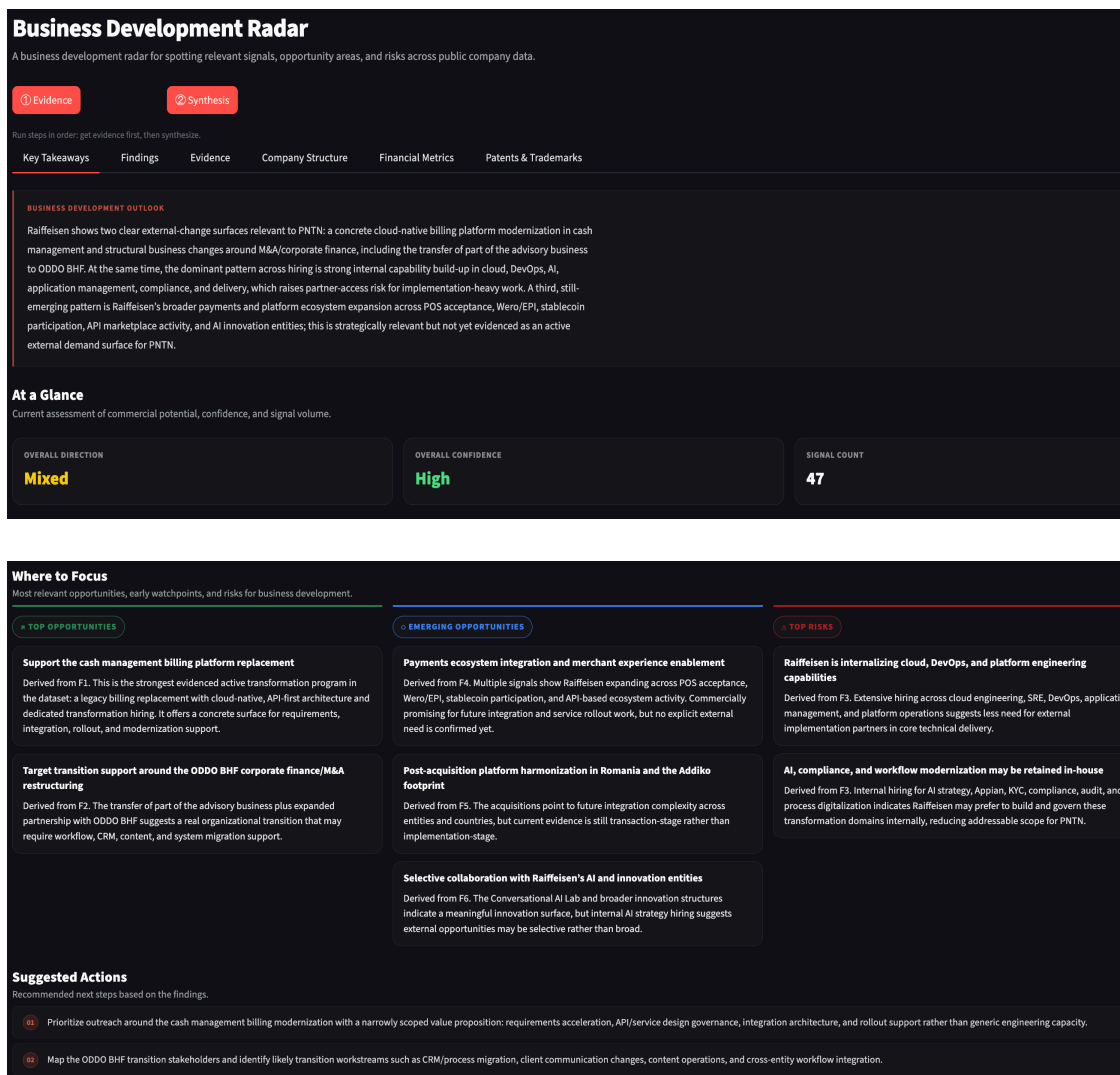


Figure 9.1 & 9.2: Key Takeaways tab (Raiffeisen, all categories selected): executive summary, overall direction and opportunity/risk focus sections

This view combines the executive summary, overall direction assessment and the three columns that present the top opportunities, emerging opportunities, and top risks. It shows how the interface places the most decision-relevant outputs at the beginning of user journey.

The Findings tab shows the grouped findings produced by LLM2. These findings merge related signals into broader observations, for example, combining a series of senior data-role hires, an innovation page update, and an R&D spend increase into a single finding about an emerging capability build. Each grouped finding is expandable and includes the overall direction and confidence behind it, making the analytical basis for the summary visible even without going to the evidence layer. As a visual example, Figure 9.3 shows the Findings tab for Raiffeisen

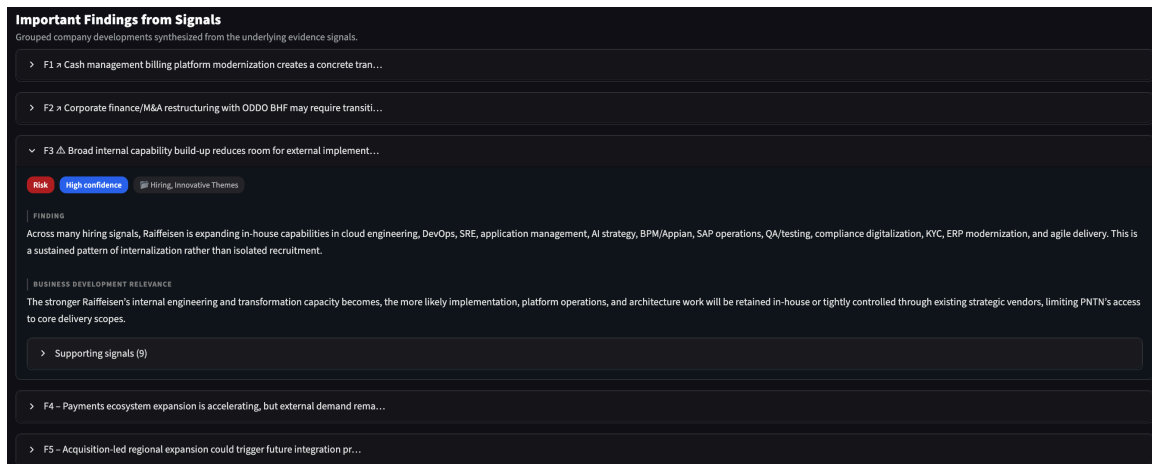


Figure 9.3: Findings tab: grouped findings produced by LLM2 with direction and confidence indicators

This view with one finding expanded makes the reasoning layer more transparent than a simple executive summary. Users can see which themes the system identified across multiple signals before deciding whether to inspect the underlying evidence.

The Evidence tab exposes the underlying LLM1-level records. Each entry shows the signal category, direction, confidence, the extracted evidence text, and the `why_it_matters_for_pntn` explanation generated at classification time. This is the layer users turn to when they want to verify a finding or investigate a specific signal area in more depth.

Three additional tabs provide structured supporting context drawn from North Data. The Company Structure tab shows organizational and ownership information, including related entities and key persons. The Financial Metrics tab presents the financial history and key performance indicators retrieved through the North Data API. The Patents & Trademarks tab surfaces intellectual property filings. We intentionally separated these tabs from the main synthesis flow. They provide factual company background that can inform interpretation, but they are not themselves signal outputs from the LLM reasoning pipeline and mixing them into the main analysis view would blur that distinction. Figure 9.4 shows the Financial Metrics tab for Epson Deutschland. This tab presents structured North Data information separately from the LLM-generated synthesis outputs.

Key Takeaways	Findings	Evidence	Company Structure	Financial Metrics	Patents & Trademarks
Financial Data					
Epson Deutschland GmbH · FY2025					
Revenue	Earnings	Equity	Return on Sales	Employees	
€371.9M	€6,366,247	€12.7M	1.7%	280	

Figure 9.4: Financial Metrics tab

The tab displays the most recent financial year's key figures including revenue, earnings, equity, return on sales, and employee count, all drawn directly from the North Data API.

Together, the six tabs give users both a fast path to actionable insight and a slower path to evidence inspection and verification. The interface does not force either path, it supports both and leaves the choice to the user depending on how much time they have and how consequential the decision in front of them is.

10. Evaluation & Results

This chapter evaluates the Business Development Radar from three perspectives: quantitative system output, qualitative usefulness, and remaining limitations. The goal of the evaluation is to assess whether the prototype successfully demonstrates the feasibility of automated signal collection, classification, synthesis, and user-facing exploration for business development.

The evaluation uses the final state of the prototype database at the end of the project. The quantitative evaluation summarizes the scale and coverage of collected data, deduplication, LLM1 classification, category coverage, embedding generation, and token reduction. The qualitative evaluation discusses whether the generated outputs were meaningful and usable for Plan.Net TechNest stakeholders during prototype testing. The error analysis then identifies recurring failure modes, followed by limitations and future work.

10.1 Quantitative Evaluation

A quantitative evaluation is crucial because it shows whether the prototype’s final database state is sufficient, reliable, and cost-effective enough to support downstream synthesis. This section examines data collection, deduplication, LLM1 classification, storage, costs, and benefits in input reduction.

10.1.1 Data Collection Results

The data collection layer produced a broad but uneven dataset across source types. In total, the crawl collected 2,330 webpages, 872 PDFs, and 1,378 North Data records, shown in Table 10.1.

Source Type	Collected Entries	Average Text Length (Characters)	Source Count	Company Count
Webpages - Total	2330	3189,1	52	21
Webpages - news/innovation	956	3043,1	28	17
Webpages - job postings	1324	3188,2	19	19
Webpages - other	50	6004,2	5	4
PDFs	872	56401536,0	27	16
PDF segments	11460	4921,6	27	16
North Data - publications	211	26697,6	19	19
North Data - other records	1167	132,2	19	19

Table 10.1: Summary of Data collection results over different source types

For webpages, job postings (1,324) and news/innovation pages (956) formed the largest groups, indicating the fact that these source types were the most consistently available across monitored companies.

The PDF processing pipeline successfully parsed 858 out of 872 collected PDFs into 11460 segments and 65499 chunks, with an average of 13.4 segments per processed PDF.

The webpage deduplication layer eliminated 537 duplicate webpages before saving, shown in table 10.2.

Deduplication Method	Total Eliminated Duplicates	News/Innovation Duplicates	Job Posting Duplicates	Other Duplicates
Fuzzy duplicate (LSH)	311	16	294	1
Exact content hash	226	163	63	0
Total	537	179	357	1

Table 10.2: Deduplication Summary

226 (42.1%) of duplicates were identified by exact content hashes and 311 (57.9%) by MinHash LSH, showing that fuzzy matching removed a substantial number of near-duplicate pages beyond exact copies, especially for duplicate job postings (294 fuzzy job duplicates). Without a deduplication layer, there would be a 23.3% (2330 to 2867) increase in collected webpages, proving how essential the deduplication layer was for the system.

In combination, the data collection results indicate that the system was able to collect and process a large amount of data robustly, in preparation for further analysis.

10.1.2 LLM1 Classification & Categorization Results

The first LLM layer classified 13,753 entries and retained a total of 1,523 useful results, consisting of 646 main and 877 weak signals, at an estimated total cost of \$32.1481, shown in Table 10.3, with relevance rates to show how often a relevant entry was identified.

Source Type	Classified Entries	Relevant Enrichments	Noise Enrichments	Relevance Rate	LLM1 Total Cost (USD)
All source types	13753	1523	12230	11,1%	\$32,1481
Webpages - total	2304	714	1590	31,0%	\$5,2515
Webpages - news/innovation	930	203	727	21,8%	\$2,1120
Webpages - job postings	1324	483	841	36,5%	\$3,0125
Webpages - other	50	28	22	56,0%	\$0,1270
PDF segments	11306	801	10505	7,1%	\$26,7107
PDFs	858	-	-	93,4%	-
North Data - publications	21	8	13	38,1%	\$0,0766
North Data - events	122	0	122	0,0%	\$0,1093

Table 10.3: General LLM1 results over different source types

The results in table 10.3 show that although webpages had more relevant signals per LLM1 input, as only 7.1% of pdf segments were classified as relevant, while webpage entries had a rate of 31,0%. Therefore, despite webpages and PDFs yielding a relatively similar amount of relevant results (714 and 801), the LLM analysis of PDFs costed 5.06 times more. This shows that for a larger dataset with higher costs, webpages should be the priority if cost becomes a concern. Lastly, although North Data Publications had a high relevance rate, the total amount of entries was limited.

A breakdown of classified categories by the LLM1 analysis can be found in tables 10.4 and 10.5.

Category	Primary Categories Found	Primary or Secondary Categories Found
Hiring	547	576
Innovative Themes	293	712
Financials	262	471
Partnerships / Acquisitions	194	303
News / Products	191	316
Legal & C-Level Updates	36	48

Table 10.4: Total amounts of classified categories over all relevant entries

Although Hiring was the most significant primary category, other categories were located more in secondary categories, creating a balanced outlook. The Legal Category was found to be much rarer than the other categories (6.6% of all primary categories). This largely indicates that the resulting dataset is balanced, yet with a key limitation of there being less content found for Legal & C-level Updates over the collected web data.

Within the context of LLM1-generated primary or secondary categories, 9 companies contained results in all six categories, while 17 of 22 companies covered at least five categories, as shown in table 10.5.

Primary / Secondary Categories Found	Company Count	Companies
6	9	B. Braun, BMW, Dertour, ECE, EON, Lufthansa, Olympus, Raiffeisen, Smart
5	8	AIDA, BNP Paribas, Berlin Airport, Epson, Greiner Bio One, Penny, Scalable Capital, Stiebel Eltron
4	0	-
3	3	EWie Einfach, IHK, Stiftung Warentest
2	1	Ehrmann
1	0	-
0	1	SPD

Table 10.5: Company Category Coverage Summary

These results show that the database was populated enough to support high-quality and detailed further analysis for 17 of the 22 companies. However, results for 5 companies are not so complete. This is partially due to the fact that we couldn't access most of the useful content for these companies due to a limited number of sources stemming from legal limitations. This is a critical limitation that users need to be aware of, as it can lead to sampling bias, making downstream synthesis may over- or underrepresent company-specific patterns simply because the available data volume differs across companies.

A more detailed outlook into all results per-company can be found in Appendix D, Table D1.

Among 1,523 useful enrichments, the dominant directions were neutral (67.2%), followed by risk (29.9%). Opportunities were scarce (3.0%), as the LLM1 prompt was engineered to be strict with identifying opportunities.

Useful Enrichments	Opportunity	Risk	Neutral
1523	45 (3.0%)	455 (29.9%)	1023 (67.2%)

Table 10.6: The Distribution of LLM1-assigned direction over all relevant entries

Overall, the quantitative evaluation of the first stage analysis indicates the creation of a balanced dataset.

10.1.3 Storage Results

In total, 44,867 raw chunk embeddings and 7,615 enrichment embeddings were generated, with a calculated combined embedding cost of \$1.3776.

Table 10.7 shows how the LLM1 stage reduced the amount of information passed to downstream synthesis.

Scenario	Entry Count	Total Input Tokens	Average Input Tokens per Entry	Multiple of Current Implementation
LLM1-generated useful-signal content	1523	318248	209,0	1,00
Only LLM1 filtering	1523	2708600	1778,5	8,51
No LLM1 step	14129	17314357	1225,4	54,41

Table 10.7: Transformation of signal sizes due to LLM1 analysis

When only LLM1-generated useful-signal content was retrieved, the input volume for LLM2 was limited to **1,523 entries** and **318,248 tokens**. In contrast, using LLM1 only as a filter while still passing raw text would require **2,708,600 tokens**, or **8.51 times** more input. Without the LLM1 step, all saved raw entries would require **17,314,357 tokens**, making the synthesis input **54.41 times** larger than in the current implementation.

These results clearly demonstrate the value of having a stage 1 analysis step, not only to filter irrelevant input, but also to extract and condense critical information. This means that the second stage LLM will feasibly understand signals within context window and recommended reasoning limits, not spend resources extracting implications, and focus on identifying cross-signal patterns.

10.2 Qualitative Evaluation

The qualitative evaluation assessed whether the prototype outputs were understandable, credible, and useful for the intended business development context. The prototype was reviewed and tested together with Plan.Net TechNest general managers Bernhard Engl and Ret Lauterbach, who evaluated both the system logic and selected company outputs.

Overall, the feedback indicated that the Business Development Radar represents a genuinely useful foundation for business development monitoring. Stakeholders emphasized that the system's value lies not in replacing human judgement, but in structuring, accelerating, and grounding early company assessments through traceable evidence. The layered design, moving from Key Takeaways to Findings and then to Evidence, was seen as especially important for trust and usability.

10.2.1 ECE Group: Confirming a Market-Level Pattern from Hiring Signals

One of the clearest validation moments came from the ECE Group risk assessment. The system identified a top risk flagging that ECE was building a self-sufficient internal analytics capability. Specifically hiring patterns across data engineering, MS Fabric, Power BI, analytics product ownership, and customer analytics leadership roles pointed toward a deliberate internalization of BI and data platform work. The generated risk read:

"Internal analytics and data-platform build-up may reduce external demand for BI and data engineering support. Roles across data engineering, MS Fabric, Power BI, analytics product ownership, and customer analytics leadership suggest ECE is building a self-sufficient analytics capability, reducing whitespace for external vendors in these domains."

Ret Lauterbach confirmed this finding directly, noting that this shift was already known at a market level and that many agencies had in fact been affected by it. What he found remarkable was that the system had derived this conclusion entirely from job descriptions that individually seem unremarkable but collectively signal a clear strategic direction. This validated a core design assumption of the radar: that weak signals distributed across multiple sources can, when combined and interpreted systematically, surface strategic patterns that would otherwise require manual monitoring to detect.

10.2.2 Olympus: Surfacing a Recently Recognized Opportunity

A second evaluation case involved Olympus, where the system generated an emerging opportunity around supply-chain, ESG, and compliance platform modernization. The finding described Olympus as expanding responsible supply-chain monitoring, ESG governance, enterprise risk management, IT security, and compliance reporting systems, with signals pointing to global supplier assessments, integrated ERM tooling with dashboards and AI, Integrity Line enhancements, and broader governance integration across legal, risk, compliance, privacy, and information security. The system categorized this as commercially interesting but not yet a clear external opening, noting that current signals suggested mature internal governance rather than an active implementation mandate.

Stakeholders recognized this finding as consistent with developments they had only recently become aware of through other channels. This was notable given the limited set of connected sources in the current prototype and demonstrated that relevant signals could already be surfaced from publicly available company-owned sources alone. The ability to trace this emerging opportunity back to specific evidence records, rather than presenting it as an unsupported conclusion, was highlighted as a key strength, particularly for building confidence in outputs before acting on them.

10.2.3 General Feedback

Beyond these two cases, the qualitative evaluation revealed clear directions for further development. Stakeholders indicated that the interface and interaction flow could be refined further, particularly around how opportunities are prioritized and explained. They also expressed interest in expanding the source base and onboarding additional companies, noting that the emergence of meaningful signals from a relatively constrained source set was unexpected and promising.

Stakeholders also discussed the longer-term potential of the radar concept beyond Plan.Net TechNest. Because the system is built around scalable business logic, reusable prompts, and traceable data pipelines, it was seen as a possible foundation that could be extended to additional agencies within the House of Communication. This perspective reinforced the project's design choice to prioritize scalability and transparency over company-specific hardcoding.

At the same time, the feedback made clear that the current system should be understood as a functional prototype rather than a finished product. While the qualitative results confirmed the usefulness of the concept and architecture, stakeholders emphasized that further refinement of opportunity logic, interface details, and source coverage would be necessary before broader operational use.

10.3 Error Analysis

The error analysis identified several recurring issues across the pipeline. The first issue concerned the boundary between opportunity, neutral, and risk classifications in LLM1. During testing, less strict prompt versions sometimes produced too many opportunity classifications, especially when a source described a digital initiative, transformation project, or new product. While these developments can be relevant for business development, not every positive company development creates an actionable opportunity for Plan.Net TechNest. The prompt was therefore tightened to require a clearer delivery surface before classifying a signal as an opportunity.

A second issue concerned the distinction between main and weak signals. Some fragments contained early indicators of possible strategic movement, such as hiring for technical roles or vague references to digital programs but did not provide enough evidence to support a strong signal classification. In these cases, the model sometimes varied between main and weak depending on phrasing and context. The three-bucket structure helped reduce the impact of this ambiguity by preserving uncertain signals as weak rather than discarding them entirely.

A third issue came from source quality. Some PDF segments, job pages, or scraped webpages contained incomplete context, boilerplate text, duplicated wording, or navigation material that could not be fully removed during preprocessing. These inputs increased the difficulty of classification and sometimes led to low-confidence or overly generic outputs. Deduplication, filtering, and prompt constraints reduced this problem, but did not eliminate it completely.

Finally, LLM2 errors were mostly upstream-dependent. When retrieval returned a narrow or incomplete evidence set, LLM2 could only synthesize what it received. This means that missing sources, conservative LLM1 classifications, or overly narrow category filters could lead to incomplete company-level summaries. The Evidence tab and confidence indicators partially mitigate this risk by allowing

users to inspect supporting signals, but the final synthesis remains dependent on the quality and completeness of the earlier pipeline stages.

10.4 Limitations

10.4.1 Prompt Sensitivity

The prototype remains sensitive to prompt wording. Small changes in the LLM1 prompt affected how strictly the model distinguished opportunities from neutral developments and how it handled borderline main/weak cases. This is a general limitation of LLM-based classification in automated pipelines: even with a fixed output schema, interpretation can shift when instructions are changed. The final prompt reduced this risk through explicit category definitions, bucket rules, direction constraints, and the `pntn_fit_check` field, but future versions should continue systematic prompt testing before deployment.

10.4.2 Cost Constraints

LLM1 used a smaller model because it had to process a high number of evidence units. This made the classification stage economically feasible, but it also meant that some nuanced cases were harder to classify reliably. The calculated cost for LLM1 analysis was 32 USD. LLM2 used a more capable model because synthesis required stronger reasoning, but this stage was only run after retrieval and therefore at much lower volume. The cost of LLM2 synthesis was not tracked systematically in the prototype, as it depends on retrieval scope and was run on demand. The staged model choice was practical for the prototype, but future scaling would still require cost monitoring, batching strategies, and further optimization of prompt length and retrieval scope.

10.4.3 Legal Constraints

Source coverage was limited by legal, technical, and practical constraints. LinkedIn was excluded because automated scraping conflicted with platform restrictions and personal-data considerations. Cloudflare-protected websites and other technically restricted sources were also excluded because bypassing access protections was outside the project's responsible scraping boundaries. These decisions kept the prototype aligned with legal and ethical constraints, but they also reduced coverage for some companies and categories. As a result, the radar should be interpreted as a structured decision-support tool based on available evidence, not as a complete representation of all relevant company activity.

10.4.4 Evaluation Scope

The evaluation was limited to the final project state and selected stakeholder testing. The project did not include a long-term longitudinal deployment, systematic user study, or statistically representative benchmark against manual analyst performance. The results therefore demonstrate feasibility and usefulness of the prototype, but they do not yet prove stable performance across time, all companies, or all business development contexts. Future evaluation should test output quality, robustness, and user value across more repeated use cases.

10.5 Conclusion

The evaluation shows that the Business Development Radar successfully demonstrated the feasibility of an automated, evidence-based signal monitoring workflow for business development. Quantitatively, the system collected and processed a substantial amount of heterogeneous company information, filtered a large share of irrelevant material, generated structured LLM1 enrichments, and reduced the input volume required for LLM2 synthesis. Qualitatively, stakeholder feedback indicated that the tool produced meaningful and recognizable outputs, especially when users moved from high-level takeaways into findings and evidence.

The prototype therefore achieved its main project objective: it showed how public company information can be transformed into structured signals, synthesized into company-level insights, and presented in a way that supports business development users. The qualitative validation, where stakeholders independently confirmed system-generated findings against their own market knowledge, provides meaningful evidence that the core signal detection and synthesis logic is working as intended. At the same time, the evaluation highlights that output quality depends heavily on source coverage, prompt design, retrieval quality, and evidence availability. The current version should therefore be understood as a strong functional prototype and foundation for further development, rather than as a fully production-ready monitoring system.

10.6 Future Work

There are four major areas for future work: incorporating additional data sources, automating the mapping process, improving management insights about the radar's reliability and coverage, and adding proactive alerting.

For the first, the current radar relies exclusively on company-owned websites. LinkedIn would be a valuable addition. We excluded direct scraping due to legal constraints, but access via an official API would unlock hiring signals through job postings, partnership and product announcements from

company pages, and executive engagement patterns that hint at internal priorities not yet visible on official channels. Third-party news sources such as industry publications and press portals would capture developments faster and more objectively, picking up developments that companies do not proactively disclose themselves. A further candidate is Dealfront, a European B2B sales intelligence platform that offers structured company data, trigger events, and GDPR-compliant contact information. We identified this integration during the project but could not be implemented within the available timeframe.

For the second, the current process of identifying and implementing websites to scrape for each company was carried out manually and was highly time-consuming. Automating this source discovery would make it significantly faster to integrate new companies. Combined with the additional data sources described above, this would also lay the groundwork for scaling the radar well beyond its current scope of 21 companies.

For the third, the data collection and processing pipeline can be fully automated, by deploying on cloud or a company datacenter. This would mean that the dataset would always be up-to-date, and able catch new signals on a daily or weekly basis. Implementing this extension would be an easy next step, as we already initialized commands that can be scheduled on cloud containers.

For the fourth, the quality of the company-level summaries depends entirely on what signals were retrieved beforehand. If relevant signals were missed, the synthesis will reflect that without flagging it. A future evaluation study could assess how accurate and well-supported these summaries actually are. In addition, a robustness check could test whether the outputs stay consistent when retrieval settings are slightly changed for the same company.

For the fifth, the radar currently relies on users actively checking for new signals. An alerting system that automatically notifies users when a high-priority signal is detected would make the tool considerably more practical in daily use, especially as the number of tracked companies grows. To support this, vector clustering could be implemented to automatically identify patterns in saved data.

Beyond these four areas, several additional improvements are worth mentioning. First, the existing vector stores could serve as the foundation for a retrieval augmented generation (RAG) based conversational interface or MCP integration, allowing users to query company signals directly through natural language rather than navigating the current tab-based interface. Second, data collection efficiency could be improved by running scrapers in parallel, while LLM1 costs would be reduced through prompt batching and prompt size optimization. Finally, incorporating structured financial figures into the analysis could open up a numbers-based layer of a company direction inference, complementing the signal-based approach currently used.

Together, these improvements would move the Business Development Radar from a functional prototype toward a continuously operating, self-updating business intelligence system.

11. Summary of Contributions

This section summarizes the main group-level and individual contributions to the project. The table distinguishes shared work packages from individual responsibilities to make the division of work transparent.

Task / Work Package	Group Members Involved	Status / Outcomes
GROUP LEVEL		
Project planning and coordination setup	Entire group; initial planning led by Azra with input from Can	Detailed project timeline and Gantt chart created, including milestones, task sequencing, and clearer member assignments, which improved coordination and planning transparency at an early stage.
Source discovery and source landscape preparation for company monitoring	Entire Group	Source discovery and mapping were structured systematically across companies and categories. This included the main source map, category coverage analysis, and early assessment of which public sources were practically useful and legally accessible for the prototype.
Source coverage analysis and category mapping	Azra Özçelik, Franziska Stannek, Valeria Vaquero	Systematically assessed all candidate sources across the five signal categories for each assigned company, evaluating whether each source contained sufficiently relevant content, was technically accessible, and was legally permissible to scrape. This included reviewing robots.txt files, terms of service, and ruling out restricted platforms such as LinkedIn resulting in a coverage map forming the foundation for the data collection.
Manual source assessment and business relevance evaluation	Azra Özçelik, Franziska Stannek, Valeria Vaquero	Selected sources were manually assessed in detail to understand what information they actually contain, what noise they produce, and how they could later support meaningful business signals.
Prompt testing across different AI tools	Azra Özçelik, Franziska Stannek, Valeria Vaquero	Manually simulated the AI classification process by testing prompts across different AI tools to assess how signals should be interpreted and categorized, informing the final system prompt design used in the BDR.
INDIVIDUAL		
Detailed project planning and timeline management & project management	Azra Özçelik	Led the preparation of the updated project plan, including the detailed Gantt chart with dates, milestones, dependencies, and clearer work-package assignments. Also managed timeline tracking and coordination of individual and team tasks during the project.

Coverage Map creation and interpretation & Final analysis and refinement of Source map	Azra Özçelik	Created the Coverage Map and used it to identify which categories were covered for which companies, where important gaps existed, and what structural patterns emerged across company types. Based on the findings finalized the Source Map by adding or refining sources.
North Data exploration and collaboration	Azra Özçelik	Contacted North Data, led the meeting process that resulted in the free API key, and guided the decision process on how North Data should be used in the radar. This included introducing the additional category “Legal & C-Level Updates” and deciding which North Data outputs should be kept, filtered, and structured for LLM processing.
Design and Development of LLM1 classification logic, prompt and all features included	Azra Özçelik	Designed and implemented the full first-stage signal-classification logic (LLM1), including prompt architecture, category structure, signal buckets, direction logic, confidence handling, and the supporting output fields needed for retrieval and interface display.
Scalable business-logic design through industry mappings	Azra Özçelik	Researched how to improve output correctness and scalability by defining rule ordering and industry-oriented mapping documents. This supported a reusable and scalable framework.
Design and Development of LLM2 company-level reasoning concept	Azra Özçelik	Designed and implemented the full second-stage reasoning logic (LLM2), including grouped findings, company-level synthesis, opportunities, risks, and output structure. Also tested LLM2 on retrieved LLM1 results and refined the logic based on observed issues in output quality and prioritization.
Design and Development of the User Interface, flow design and interaction logic	Azra Özçelik	Designed the website/UI flow and later developed the final interface logic from a business-user perspective. The final UI structure was shaped around how Plan.Net TechNest wanted to navigate the tool independently, especially in terms of moving from takeaways to findings to evidence.
Biweekly supervisor meeting slide preparation	Azra Özçelik	Prepared recurring progress meeting slides
Result evaluation feedback sessions with Plan.Net Technest	Azra Özçelik	Held 2 meetings with Plan.Net TechNest to explain the final tool, logic and flow. During these meetings, the results have been evaluated, and feedback has been received for the Section 10. This shaped the planning of future work for our prototype.
Authorship of Sections 5.1, 6.0-6.6, 8.0-8.4, 9.0-9.4, 10.0, 10.2-10.5 Co-Authorship of Section 4	Azra Özçelik	
Repository set up	Olaf Taczowski	Set up the initial GitHub repository and defined the team's collaboration workflow, including Python virtual environments and a requirements.txt-based dependency management system to ensure consistent and conflict-free package management across the project.
Development of first web scraping workflow using CSS	Olaf Taczowski	Developed the workflow for accessing web data using CSS selectors, identifying browser-based tooling to support selector extraction and webpage inspection.
Defined rules and logic for scraping pdf's from company financial sites	Olaf Taczowski	Implemented scraping logic for financial websites based on webpage block structures and created internal models for storing the extracted information.
Integrating North Data into scraping pipeline	Olaf Taczowski	Integrated North Data into the scraping pipeline by accessing and processing its most relevant API endpoints. This required cleaning and structuring the retrieved data, grouping key financial statistics for

		efficient downstream LLM access, and creating the necessary database tables for storing North Data outputs.
Initial cleaning of the database	Olaf Taczkowski	Analyzed the database for duplicate entries using the pg_trgm extension and reduced duplication through scraper-level filtering and link-cleaning mechanisms.
Design and implementation of PDF parsing tool.	Olaf Taczkowski	Design and implemented the PDF-processing pipeline using IBM Docling. This involved evaluating different parsing strategies and library configurations, testing both local and remote image-description models, and integrating GPT-based image descriptions into the processing workflow.
Developing chunking framework for PDF's	Olaf Taczkowski	Implemented document chunking using Docling's chunking framework.
Filtered job postings	Olaf Taczkowski	Implemented filtering for job postings to only include EU-based vacancies.
Authorship of sections 5.3, 5.5, 5.6 of the report	Olaf Taczkowski	
Development of a modular web scraping framework with Scrapy	Can Öcek	Identified Sitemap crawling and RSS feed crawling as feasible methods, implemented alongside recursive crawling, based on an extensive modular framework to filter and extract desired content. Manually populated configuration files for most sources crawled using these methods. Provided various edge case fixes
Implementation of dynamic crawling with Playwright	Can Öcek	Implemented scrapy-playwright to crawl dynamically-loaded websites, with custom item interactions for 4 specific sources
Trafilatura implementation for HTML parsing	Can Öcek	Identified and implemented the library Trafilatura as an alternative to CSS selectors for scalably parsing text and relevant metadata like date and titles from raw HTML sources
Research on external dataset-supported LLM systems	Can Öcek	Researched AI systems that utilize a large dataset to support reasoning to make grounded system design decision. Identified and used PostgreSQL and pgvector as a good way to implement a hybrid database framework to support vector retrieval
Design and implementation of the database schemas	Can Öcek	Designed and implemented SQL Database schema for holding data for all entries, including webpages, enrichments/analyses, embeddings, except for northdata-related tables.
Hash and MinHash deduplication of sources	Can Öcek	Implemented regular hashing and MinHash LSH fuzzy deduplication for reducing collected duplicate webpage content
Aggregation of all processes	Can Öcek	Set up Command Line Interface endpoints for all processes and crawls, enabling easy and extendable way to manually populate the database. Integrated processes like LLM to the database flow of data.
Database integration	Can Öcek	Set up company GitLab repository, established local Docker and for testing the database and processes, and Docker containers for local testing of scheduling, in addition to setting up a Neon database later to support the deployed user interface.
Retrieval script to fetch entries from the database	Can Öcek	Implemented both regular SQL and multi-vector based search over the database to fetch the desired group of entries from the database, to be ingested into the second LLM stage.
Test UI deployment and integration into Database	Can Öcek	Connected the webapp UI into retrieval flows from the database, established a remote connection for testing with a database, set up hosting in the Streamlit community cloud. Additionally, ensured functionality of the connections with the backend and were represented on the UI

PDF segmentation	Can Öcek	Added functionality to merge small pdf chunks by headings to create LLM-efficient segments
Data Evaluation	Can Öcek	Wrote scripts to query the database to form sheets to support data analysis to ensure quality and support quantitative evaluation
Authorship of Sections 5.2, 5.4, 7, 10.1 of the report Co-Authorship of Sections 4, 10.6	Can Öcek	
Legal framework research and documentation	Valeria and Franziska	Conducted academic literature review to assess the legality of web scraping in the EU context, resulting in the legal framework in Chapter 3 which defined what could and could not be scraped and guided all source decisions throughout the project.
Literature Review	Valeria and Franziska	Jointly researched and wrote the literature review, covering key academic sources across web scraping methods, data extraction techniques, legal considerations and LLM-based business insight generation, providing the theoretical foundation for the project.
Source Map Contribution	Valeria and Franziska	Identified and verified suitable sources for each signal across assigned companies, reviewed robots.txt and terms of service, assessed content relevance and peer reviewed sources selected by other team members.
Technical Support and cross-team collaboration	Valeria and Franziska	Researched and responded to technical questions from the development team throughout the project, supporting system development in parallel with research tasks.
System testing and evaluation	Valeria and Franziska	Tested the final BDR across four companies (Olympus, Raiffeisen, Epson and Stiebel Eltron), evaluated whether the tool fulfilled its intended purpose, and communicated findings to the team.
Report structuring and preparation	Valeria, Franziska and Azra	Jointly responsible for defining the structure of the report and coordinating its preparation
Authorship of sections 2.0-2.4, 3.0, 3.1, 3.2, 3.5, 3.6, 5.7.1, 10.3.4, 10.6	Valeria Vaquero	
Authorship of sections 1.0-1.5, 2.5-2.8, 3.3, 3.4, 3.7, 3.8	Franziska Stannek	

12. References

- Adnan, K., & Akbar, R. (2019). Limitations of information extraction methods and techniques for heterogeneous unstructured big data. *International Journal of Engineering Business Management*, 11, 1–23. <https://doi.org/10.1177/1847979019890771>
- Ansoff, H. I. (1975). Managing strategic surprise by response to weak signals. *California Management Review*, 18(2), 21–33. <https://doi.org/10.2307/41164635>
- Badmus, O., Rajput, S. A., Arogundade, J. B., & Williams, M. (2024). AI-driven business analytics and decision making. *World Journal of Advanced Research and Reviews*, 24(1), 616–633. <https://doi.org/10.30574/wjarr.2024.24.1.3093>
- Barbaresi, A. (2021). Trafilatura: A web scraping library and command-line tool for text discovery and extraction. *Proceedings of the Joint Conference of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations*, 122–131. <https://doi.org/10.18653/v1/2021.acl-demo.15>
- Bereczki, T., & Liber, Á. (2024). The state of web scraping in the EU. *IAPP*. <https://iapp.org/news/a/the-state-of-web-scraping-in-the-eu>
- Broder, A. Z. (1997). On the resemblance and containment of documents. In *Proceedings of Compression and Complexity of Sequences 1997* (pp. 21–29). IEEE. <https://doi.org/10.1109/SEQUEN.1997.666900>
- Brown, M. A., Gruen, A., Maldoff, G., Messing, S., Sanderson, Z., & Zimmer, M. (2025). Web scraping for research: Legal, ethical, institutional, and scientific considerations. *Big Data & Society*, 12(1), 1–17. <https://doi.org/10.1177/20539517251381686>
- Chang, Z. (2022). A survey of modern crawler methods. In *Proceedings of the 6th International Conference on Control Engineering and Artificial Intelligence* (pp. 110–114). Association for Computing Machinery. <https://doi.org/10.1145/3522749.3523076>
- Chaudhuri, S., Dayal, U., & Narasayya, V. (2011). An overview of business intelligence technology. *Communications of the ACM*, 54(8), 88–98. <https://doi.org/10.1145/1978542.1978562>
- Chourey, L. (2024, December 4). Steering LLMs toward desired outputs with prompt engineering. *ISACA Journal*, 6. <https://www.isaca.org/resources/isaca-journal/issues/2024/volume-6/steering-llms-toward-desired-outputs-with-prompt-engineering>
- Citus Data. (n.d.). *pg_cron: Run periodic jobs in PostgreSQL* [Computer software]. GitHub. Retrieved May 22, 2026, from https://github.com/citusdata/pg_cron

Cloudflare. (2024). *How to prevent web scraping*. <https://www.cloudflare.com/learning/ai/how-to-prevent-web-scraping/>

Docling Project. (n.d.). *Docling documentation*. <https://docling-project.github.io/docling/>

Docker. (n.d.). *What is a container?* Retrieved May 22, 2026, from <https://www.docker.com/resources/what-container/>

Elnashar, A., White, J., & Schmidt, D. C. (2025). Enhancing structured data generation with GPT-4o: Evaluating prompt efficiency across prompt styles. *Frontiers in Artificial Intelligence*, 8, 1558938. <https://doi.org/10.3389/frai.2025.1558938>

Ferrara, E., De Meo, P., Fiumara, G., & Baumgartner, R. (2014). Web data extraction, applications and techniques: A survey. *Knowledge-Based Systems*, 70, 301–323. <https://doi.org/10.1016/j.knosys.2014.07.007>

Foerderer, J. (2023). Interfirm data exchange and market intelligence: Evidence from web scraping. *Management Science*, 69(10), 6287–6306. <https://doi.org/10.1287/mnsc.2023.4765>

Gandhi, R., Yadav, S., & Sharma, P. (2023). ETL data pipeline to analyze scraped data. In *Proceedings of InCITE 2023*. Springer. <https://doi.org/10.1007/978-981-99-1xxx>

Glez-Peña, D., Lourenço, A., López-Fernández, H., Reboiro-Jato, M., & Fdez-Riverola, F. (2014). Web scraping technologies in an API world. *Briefings in Bioinformatics*, 15(5), 788–797. <https://doi.org/10.1093/bib/bbt026>

GroupBWT. (2026). *Web scraping in data science: Architecture & ML pipelines*. <https://groupbwt.com/blog/web-scraping-in-data-science/>

Haans, R. F. J., & Mertens, M. J. (2026). The internet never forgets: A four-step scraping tutorial for longitudinal organizational website data. *Organizational Research Methods*. <https://doi.org/10.1177/10944281251>

HCI International. (2023). Applications of large language models in business analytics — Exemplary use cases in data preparation tasks. In *Proceedings of HCI International 2023*, LNCS 14057. <https://doi.org/10.1007/978-3-031-35763-3>

Hiltunen, E. (2008). The future sign and its three dimensions. *Futures*, 40(3), 247–260. <https://doi.org/10.1016/j.futures.2007.08.021>

IAPP. (2019). *LinkedIn v. hiQ and the trans-Atlantic privacy divide*. <https://iapp.org/news/a/linkedin-v-hiq-and-the-transatlantic-privacy-divide>

- Integrate.io. (2026). *Data transformation: Explained*. <https://www.integrate.io/blog/data-transformation/>
- Konstantinou, I., & Martin, S. (2023). Establishing a framework for the ethical and legal use of web scrapers. *International Journal of Law and Information Technology*, 31(2), 112–134. <https://doi.org/10.1093/ijlit/eaad006>
- Kosala, R., & Blockeel, H. (2000). Web mining research: A survey. *ACM SIGKDD Explorations Newsletter*, 2(1), 1–15. <https://doi.org/10.1145/360402.360406>
- Kotte, V. (2025). PromptPort: A reliability layer for cross-model structured extraction. *arXiv*. <https://arxiv.org/abs/2601.06151>
- Kotte, V. (2025). The liabilities of robots.txt. *Computer Law & Security Review*. <https://doi.org/10.1016/j.clsr.2025.xx>
- Krotov, V., & Silva, L. (2018). Legality and ethics of web scraping. *Proceedings of the 24th Americas Conference on Information Systems*. <https://aisel.aisnet.org/amcis2018/DataScience/Presentations/17/>
- Lesca, H., & Lesca, N. (2011). *Weak signals for strategic intelligence: Anticipation tool for managers*. Wiley-ISTE. <https://doi.org/10.1002/9781118619506>
- Leskovec, J., Rajaraman, A., & Ullman, J. D. (2020). *Mining of massive datasets* (3rd ed.). Cambridge University Press. <https://www.mmms.org/>
- LinkedIn. (2024). *Prohibited software and extensions*. <https://www.linkedin.com/help/linkedin/answer/a1341387>
- LinkdAPI. (2025). *Best Proxycurl alternatives for LinkedIn data*. <https://linkdapi.com/blog/proxycurl-alternatives>
- Livathinos, N., Auer, C., Lysak, M., Nassar, A., Dolfi, M., Vagenas, P., Berrospi, C., Omenetti, M., Dinkla, K., Kim, Y., Gupta, S., Teixeira de Lima, R., Weber, V., Morin, L., Meijer, I., Kuropiatnyk, V., & Staar, P. W. J. (2024). Docling technical report. *arXiv preprint arXiv:2408.09869*. <https://arxiv.org/abs/2408.09869>
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.
- Microsoft. (n.d.). *Playwright*. Retrieved May 22, 2026, from <https://playwright.dev/>
- Morgan Lewis. (2024). *EU regulator adopts restrictive GDPR position on data scraping*. <https://www.morganlewis.com/pubs/2024/eu-regulator-adopts-restrictive-gdpr-position-on-data-scraping>

- Mühlroth, C., & Grottke, M. (2021). Innovation signals: Leveraging machine learning to separate noise from news. *PLoS ONE*, 16(5), e0251765. <https://doi.org/10.1371/journal.pone.0251765>
- Mühlroth, C., Kölbl, L., & Grottke, M. (2023). Innovation signals: Leveraging machine learning to separate noise from news. *Scientometrics*, 128(5), 2649–2676. <https://doi.org/10.1007/s11192-023-04672-y>
- NorthData. (n.d.-a). *North Data – Company information platform*. <https://www.northdata.com/>
- NorthData. (n.d.-b). *Data API user guide*. GitHub. Retrieved May 22, 2026, from <https://github.com/northdata/api/blob/master/doc/data-api-userguide/data-api-userguide.md>
- NYU Libraries. (2025). *What large language models are — Machines and society research guide*. New York University. <https://guides.nyu.edu/machinesandsociety/lms>
- Okoone. (2026, February 9). *What you need to know before scraping EU websites*. <https://www.okoone.com/spark/strategy-transformation/what-you-need-to-know-before-scraping-eu-websites/>
- OpenAI. (n.d.). *text-embedding-3-large*. OpenAI API. Retrieved May 22, 2026, from <https://developers.openai.com/api/docs/models/text-embedding-3-large>
- pgvector. (n.d.). *pgvector: Open-source vector similarity search for Postgres* [Computer software]. GitHub. Retrieved May 22, 2026, from <https://github.com/pgvector/pgvector>
- Pichiyan, V., Arumugam, S., & Krishnan, R. (2024). Web scraping using natural language processing: Exploiting unstructured text for data extraction and analysis. *Procedia Computer Science*, 233, 45–54. <https://doi.org/10.1016/j.procs.2024.xx>
- PostgreSQL Global Development Group. (n.d.). *F.35. pg_trgm — Support for similarity of text using trigram matching*. PostgreSQL Documentation. Retrieved May 22, 2026, from <https://www.postgresql.org/docs/current/pgtrgm.html>
- Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 (General Data Protection Regulation). *Official Journal of the European Union*, L 119, 1–88.
- Regulation (EU) 2022/868 of the European Parliament and of the Council of 30 May 2022 (Data Governance Act). *Official Journal of the European Union*, L 152, 1–44.
- Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 (EU AI Act). *Official Journal of the European Union*.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language*

Processing and the 9th International Joint Conference on Natural Language Processing (pp. 3982–3992). Association for Computational Linguistics. <https://doi.org/10.18653/v1/D19-1410>

RSS Advisory Board. (2009). *RSS 2.0 specification*. <https://www.rssboard.org/rss-specification>

Sahoo, P., Singh, A. K., Saha, S., Jain, V., Mondal, S., & Chadha, A. (2024). A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv*. <https://doi.org/10.48550/arXiv.2402.07927>

Scrapy developers. (n.d.). *Architecture overview*. Scrapy documentation. Retrieved May 22, 2026, from <https://docs.scrapy.org/en/latest/topics/architecture.html>

Scrapy-Playwright contributors. (n.d.). *scrapy-playwright: Playwright integration for Scrapy* [Computer software]. GitHub. Retrieved May 22, 2026, from <https://github.com/scrapy-plugins/scrapy-playwright>

Şeker, Ş. E. (2024). Editorial: Large language models in work and business. *Frontiers in Artificial Intelligence*, 7, 1516832. <https://doi.org/10.3389/frai.2024.1516832>

Sitemaps.org. (n.d.). *Sitemaps XML format*. Retrieved May 22, 2026, from <https://www.sitemaps.org/protocol.html>

Soumyals. (2025). *How LLMs are revolutionizing data analysis: From text to insights*. Medium. <https://medium.com/@soumyals/how-llms-are-revolutionizing-data-analysis>

Tirado, J. M., Serban, O., Guo, Q., & Yoneki, E. (2016). *Web data knowledge extraction*. University of Cambridge Computer Laboratory. <https://arxiv.org/pdf/1603.07534>

Turing. (2026). *How LLMs are changing the face of business analytics*. <https://www.turing.com/kb/llms-in-business-analytics>

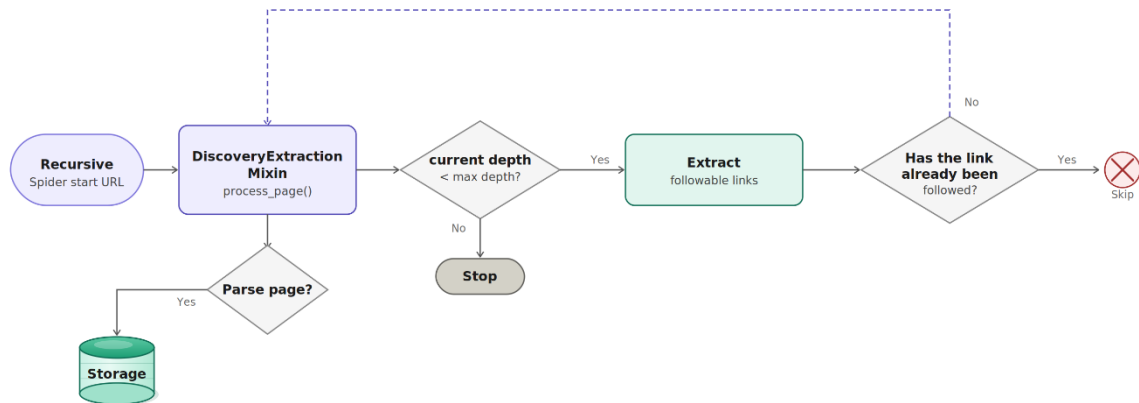
van der Lans, R. F. (2012). *Data virtualization for business intelligence systems: Revolutionizing data integration for data warehouses*. Morgan Kaufmann

Vertsel, A., & Rumiantsau, M. (2024). Hybrid LLM/rule-based approaches to business insights generation from structured data. *arXiv*. <https://doi.org/10.48550/arXiv.2404.15604>

Wu, H. (2019). Crawler technology based on Scrapy framework. *International Journal of Advanced Network, Monitoring and Controls*, 4(3), 25–31. <https://doi.org/10.21307/ijanmc-2019-056>

13. Appendix

Appendix A: Recursive Web Crawling Flowchart



Appendix B: Playwright Item Interaction Logic

This appendix summarizes the Playwright-based interaction logic used for dynamic source pages. The flows are separated because each source required a different loading and extraction strategy.

Appendix B.1: Paginated Link Discovery Flow

Sources: Stiebel Eltron Press, Dertour Jobs

Input: Initial source page

Output: Extracted content from unique discovered links

1. Load the initial source page.
2. Accept cookies by clicking the cookie acceptance button.
3. Store the current HTML snapshot.
4. While the “next” pagination button is available and active:
 - a. Click the “next” button.
 - b. Wait for the new page content to load.
 - c. Store the updated HTML snapshot.
5. Extract all relevant links from the collected HTML snapshots.
6. Remove duplicate links.
7. For each unique link:
 - a. Follow the link.
 - b. Extract the page content using the Discovery Extraction Mixin.
8. Store the extracted content for further processing.

Appendix B.2: Load More and Accordion Expansion Flow

Source: Olympus Jobs Board

Input: Olympus jobs page

Output: Extracted content from job posting blocks

1. Load the Olympus jobs page.
2. Accept cookies by clicking the cookie acceptance button.
3. Initialize the load-more click counter at 0.
4. While the “load more” button is available and the counter is below 30:
 - a. Click the “load more” button.
 - b. Wait for additional job postings to load.
 - c. Increase the click counter by 1.
5. Identify all accordion elements that contain job postings.
6. For each accordion element:
 - a. Open the accordion.
 - b. Wait for the expanded content to become available.
7. After all accordions have been opened, extract the visible and retained HTML content.
8. Use custom CSS selectors to extract each job posting block.
9. Store the extracted job posting content for further processing.

Appendix B.3: Investor PDF Collection Flow

Source: BNP Paribas Investors Page

Input: BNP Paribas investor page

Output: Collected PDF item links

1. Load the BNP Paribas investor page.
2. Initialize the load-more click counter at 0.
3. While the “load more” button is available and the counter is below 9:
 - a. Click the “load more” button.
 - b. Wait for additional investor items to load.
 - c. Increase the click counter by 1.
4. Identify all loaded investor document items.
5. For each document item:
 - a. Check the document format.
 - b. If the item links to a CSV file, skip it.
 - c. If the item links to a PDF file, collect the PDF link.
 - d. Stop collection once the predefined data limit has been reached.
6. Store the collected PDF links for further processing.

Appendix C: Deduplication Logic

This appendix provides a simplified representation of the fuzzy duplicate detection process used for webpage entries. The process identifies near-duplicate content by converting webpage text into compact MinHash signatures and comparing only likely duplicate candidates.

Input: Raw webpage text

1. Clean and normalize the raw webpage text.
2. Convert the cleaned text into lowercase word tokens.
3. Create 5-word shingles from the token sequence.
4. Generate a MinHash signature with 128 values from the shingles.
5. Split the MinHash signature into 16 bands of 8 values each.
6. Hash each band and store the band hashes in the database.
7. When a new webpage is inserted:
 - a. Apply the same cleaning, tokenization, shingling, and MinHash process.
 - b. Search the database for webpages that share at least one identical band hash.
 - c. Mark matching webpages as candidate duplicates.
8. Compare the new webpage only against the candidate duplicates using their full MinHash signatures.
9. Estimate similarity by calculating the proportion of matching values in the 128-value signatures.
10. If the highest similarity score is at least 0.80:
 classify the webpage as a fuzzy duplicate.
 Else:
 store the webpage as a unique entry.

The band-based lookup reduces the number of full comparisons required, since only webpages with at least one matching band are evaluated in detail. This allows near-duplicate detection to remain computationally efficient even when many webpages are stored.

Appendix D: Evaluation Details

Table D1: Full Company Breakdown

Total Entry count refers to all webpage, pdf segment, and North Data publications.

Company	Total Entries	Source Count	Relevant Entries	Relevant Entry Types	Primary Categories	Primary Category Counts
BMW	3352	7	305	webpages: 171; pdf segments: 134	6	Financials: 31; News / Products: 30; Partnerships / Acquisitions: 12; Hiring: 131; Innovative Themes: 100; Legal & C-Level Updates: 1
Olympus	2010	6	255	pdf segments: 237; webpages: 18	6	Financials: 80; News / Products: 46; Partnerships / Acquisitions: 48; Hiring: 21; Innovative Themes: 58; Legal & C-Level Updates: 2

Lufthansa	839	4	145	webpages: 84; pdf segments: 61	6	Financials: 33; News / Products: 14; Partnerships / Acquisitions: 14; Hiring: 52; Innovative Themes: 27; Legal & C-Level Updates: 5
Raiffeisen	1086	5	114	pdf segments: 67; webpages: 40; North Data publications: 7	6	Financials: 36; News / Products: 6; Partnerships / Acquisitions: 10; Hiring: 32; Innovative Themes: 23; Legal & C- Level Updates: 7
B. Braun	463	4	92	webpages: 70; pdf segments: 22	6	Financials: 6; News / Products: 3; Partnerships / Acquisitions: 3; Hiring: 67; Innovative Themes: 8; Legal & C-Level Updates: 5
Smart	437	3	74	pdf segments: 68; webpages: 6	6	Financials: 16; News / Products: 5; Partnerships / Acquisitions: 42; Hiring: 6; Innovative Themes: 3; Legal & C- Level Updates: 2
BNP Paribas	1107	2	87	pdf segments: 77; webpages: 10	5	Financials: 24; News / Products: 15; Partnerships / Acquisitions: 31; Hiring: 2; Innovative Themes: 15
ECE	242	4	52	webpages: 28; pdf segments: 24	5	News / Products: 13; Partnerships / Acquisitions: 4; Hiring: 17; Innovative Themes: 17; Legal & C-Level Updates: 1
Epson	171	3	49	webpages: 49	5	Financials: 1; News / Products: 8; Partnerships / Acquisitions: 12; Hiring: 24; Innovative Themes: 4
Dertour	140	5	44	webpages: 42; pdf segments: 2	5	Financials: 1; News / Products: 7; Partnerships / Acquisitions: 16; Hiring: 19; Innovative Themes: 1
Greiner Bio One	190	2	40	pdf segments: 30; webpages: 10	5	Financials: 4; News / Products: 4; Partnerships / Acquisitions: 1; Hiring: 18; Innovative Themes: 13
Berlin Airport	240	5	52	pdf segments: 30; webpages: 22	4	Financials: 8; News / Products: 15; Hiring: 13; Innovative Themes: 16

Scalable Capital	79	4	32	webpages: 24; pdf segments: 8	4	Financials: 4; News / Products: 7; Hiring: 16; Legal & C-Level Updates: 5
AIDA	2742	1	22	pdf segments: 22	4	Financials: 9; News / Products: 5; Hiring: 2; Innovative Themes: 6
EON	277	2	105	webpages: 89; pdf segments: 16	3	Financials: 9; Hiring: 89; Legal & C-Level Updates: 7
Stiebel Eltron	119	3	23	webpages: 22; North Data publications: 1	3	News / Products: 5; Hiring: 17; Legal & C-Level Updates: 1
IHK	319	5	6	pdf segments: 3; webpages: 3	3	News / Products: 3; Hiring: 1; Innovative Themes: 2
Penny	87	3	17	webpages: 17	2	News / Products: 1; Hiring: 16
EWie Einfach	26	1	5	webpages: 5	2	News / Products: 4; Partnerships / Acquisitions: 1
Ehrmann	34	2	3	webpages: 3	1	Hiring: 3
Stiftung Warentest	149	4	1	webpages: 1	1	Hiring: 1
SPD	25	3	0		0	